

BEAR: Towards Beam-Search-Aware Optimization for Recommendation with Large Language Models

Wei Qin Yang^{†‡}
Zhejiang University
Hangzhou, China
tinysnow@zju.edu.cn

Bohao Wang^{†‡}
Zhejiang University
Hangzhou, China
bohao.wang@zju.edu.cn

Zhenxiang Xu^{†‡}
Zhejiang University
Hangzhou, China
zhenxiangxu@zju.edu.cn

Jiawei Chen^{*†‡§}
Zhejiang University
Hangzhou, China
sleepyhunt@zju.edu.cn

Shengjia Zhang^{†‡}
Zhejiang University
Hangzhou, China
shengjia.zhang@zju.edu.cn

Jingbang Chen^{†‡}
The Chinese University of Hong Kong
Shenzhen, China
chenjb@cuhk.edu.cn

Canghong Jin^{†§}
Hangzhou City University
Hangzhou, China
jinch@zucc.edu.cn

Can Wang^{†§}
Zhejiang University
Hangzhou, China
wcan@zju.edu.cn

Abstract

Recent years have witnessed a rapid surge in research leveraging Large Language Models (LLMs) for recommendation. These methods typically employ supervised fine-tuning (SFT) to adapt LLMs to recommendation scenarios, and utilize beam search during inference to efficiently retrieve B top-ranked recommended items. However, we identify a critical *training-inference inconsistency*: while SFT optimizes the overall probability of positive items, it does not guarantee that such items will be retrieved by beam search even if they possess high overall probabilities. Due to the greedy pruning mechanism, beam search can prematurely discard a positive item once its prefix probability is insufficient.

To address this inconsistency, we propose **BEAR (Beam-Search-Aware Regularization)**, a novel fine-tuning objective that explicitly accounts for beam search behavior during training. Rather than directly simulating beam search for each instance during training, which is computationally prohibitive, BEAR enforces a relaxed *necessary condition*: each token in a positive item must rank within the top- B candidate tokens at each decoding step. This objective effectively mitigates the risk of incorrect pruning while incurring negligible computational overhead compared to standard SFT. Extensive experiments across four real-world datasets demonstrate

that BEAR significantly outperforms strong baselines. Code will be released upon acceptance.

CCS Concepts

• **Information systems** → **Recommender systems**.

Keywords

Recommender Systems; Large Language Models; Beam Search

ACM Reference Format:

Wei Qin Yang, Bohao Wang, Zhenxiang Xu, Jiawei Chen, Shengjia Zhang, Jingbang Chen, Canghong Jin, and Can Wang. 2026. BEAR: Towards Beam-Search-Aware Optimization for Recommendation with Large Language Models. In *Proceedings of arXiv Preprint (Preprint)*. ACM, New York, NY, USA, 12 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Large Language Models (LLMs) have exhibited remarkable abilities in natural language understanding [6], generation [44], and reasoning [60], achieving notable successes across a wide range of domains [46, 65, 75]. These advances have sparked growing interest in leveraging LLMs as the backbone for recommender systems, a.k.a. LLM-based RS [12, 17, 23, 30, 47, 72]. This paradigm typically reformulates recommendation as a natural language task, involving three key stages: (i) *Prompt construction*, converting a user’s historical interactions into a structured textual prompt to guide the LLMs; (ii) *Supervised fine-tuning* (SFT), pairing prompts with target positive items as training instances to adapt the LLMs for recommendation; and (iii) *Inference*, generating B recommended items via decoding strategies, and selecting the top- K ranked items as final recommendations [4, 14, 33]. Compared with conventional recommenders, LLM-based RS can perform fine-grained preference modeling [31], understand complex contextual information [5], and enhance recommendation interpretability [16], making them a promising and emerging direction.

BEAR: 面向大语言模型推荐的束搜索感知优化

Wei Qin Yang^{†‡}浙
江大学 中国杭州
ti
nysnow@zju.edu.cn

Bohao Wang^{†‡}浙江大
学 中国杭州
b
ohao.wang@zju.edu.cn

Zhenxiang Xu^{†‡}浙江
大学 中国杭州
z
henxiangxu@zju.edu.cn

Jiawei Chen^{*†‡§}浙
江大学 中国杭州
sl
eepyhunt@zju.edu.cn

Shengjia Zhang^{†‡}浙江大
学 中国杭州
shengjia.zhang@zju.edu.cn

Jingbang Chen^{†‡}香港中文大
学 (深圳), 中国
chenjb@cuhk.edu.cn

Canghong Jin^{†§}杭
州城市学院, 中国杭州
jinch@zucc.edu.cn

Can Wang^{†§}浙
江大学, 中国杭州
wcan@zju.edu.cn

摘要

近年来，利用大语言模型（LLM）进行推荐的研究迅速激增。这些方法通常采用监督微调（SFT）使 LLM 适应推荐场景，并在推理阶段利用束搜索（beam search）高效检索 B 排名靠前的推荐物品。然而，我们发现了一个关键的训练 - 推理不一致问题：虽然 SFT 优化了正样本物品的整体概率，但即使这些物品具有较高的整体概率，也无法保证它们能被束搜索检索到。由于贪婪剪枝机制，一旦某个正样本物品的前缀概率不足，束搜索可能会过早将其丢弃。为解决这一不一致性问题，我们提出了**BEAR (Beam-Earch-Aware Regularization)**，即束搜索感知正则化），这是一种新颖的微调目标，能够在训练过程中显式地考虑束搜索的行为。鉴于在训练期间对每个实例直接模拟束搜索的计算成本过高，BEAR 施加了一个宽松的必要条件：正样本物品中的每个 token 在每一步解码时都必须位列前 B 个候选 token 之中。该目标有效降低了错误剪枝的风险，同时与标准 SFT 相比，其计算开销几乎可以忽略不计。在四个真实世界数据集上进行的大量实验表明，

*通讯作者。
†浙江大学区块链与数据安全全国重点实验室。
‡浙江大学计算机科学与技术学院。§杭州高新区（滨江）区块链与数据安全研究院。

允许为个人或课堂使用制作本作品全部或部分的数字或硬拷贝，无需付费，前提是拷贝不得用于盈利或商业优势，且拷贝须包含本通知及首页上的完整引用。本作品中由作者以外各方拥有的组件版权必须得到尊重。允许注明出处地摘录。若要另行复制、重新发布、发布到服务器或重新分发至列表，则需事先获得具体许可和/或支付费用。请访问 permissions@acm.org 申请许可。预印本，中国杭州© 2026 版权归所有者/作者所有。出版权已授权给 ACM。ACM ISBN XXX-X-XXXX-XXXX-X/XX/XX<https://doi.org/XXXXXXX.XXXXXXX>

BEAR 显著优于强大的基线模型。代码将在论文被录用后发布。

CCS 概念

• **信息系统** → **推荐系统**。

关键词

推荐系统；大语言模型；束搜索

ACM 参考格式:

Wei Qin Yang, Bohao Wang, Zhenxiang Xu, Jiawei Chen, Shengjia Zhang, Jingbang Chen, Canghong Jin 和 Can Wang. 2026 年。BEAR: 面向大语言模型推荐的束搜索感知优化。载于《arXiv 预印本论文集》（预印本）。ACM, 美国纽约州纽约市, 12 页。<https://doi.org/XXXXXXX.XXXXXXX>

1 引言

大型语言模型（LLM）在自然语言理解 [6], 生成 [44],与推理 [60], 方面展现出卓越能力 [46, 65, 75], 在众多领域取得了显著成功。这些进展引发了人们日益浓厚的兴趣，致力于将 LLM 作为推荐系统的核心骨干，即基于 LLM 的推荐系统（LLM-based RS）[12, 17, 23, 30, 47, 72]。该范式通常将推荐任务重构为自然语言任务，涉及三个关键阶段：(i)提示构建，即将用户的历史交互转化为结构化的文本提示，以引导 LLM；(ii)监督微调(SFT)，将提示与目标正样本物品配对作为训练实例，以使 LLM 适配推荐任务；以及 (iii) 推理，通过解码策略 B 生成推荐物品，并选取排名靠前的 K 物品作为最终推荐结果 [4, 14, 33]。与传统推荐系统相比，基于 LLM 的推荐系统能够执行细粒度的偏好建模 [31], 理解复杂的上下文信息 [5],并提升推荐的可解释性 [16],，使其成为一个充满前景的新兴方向。

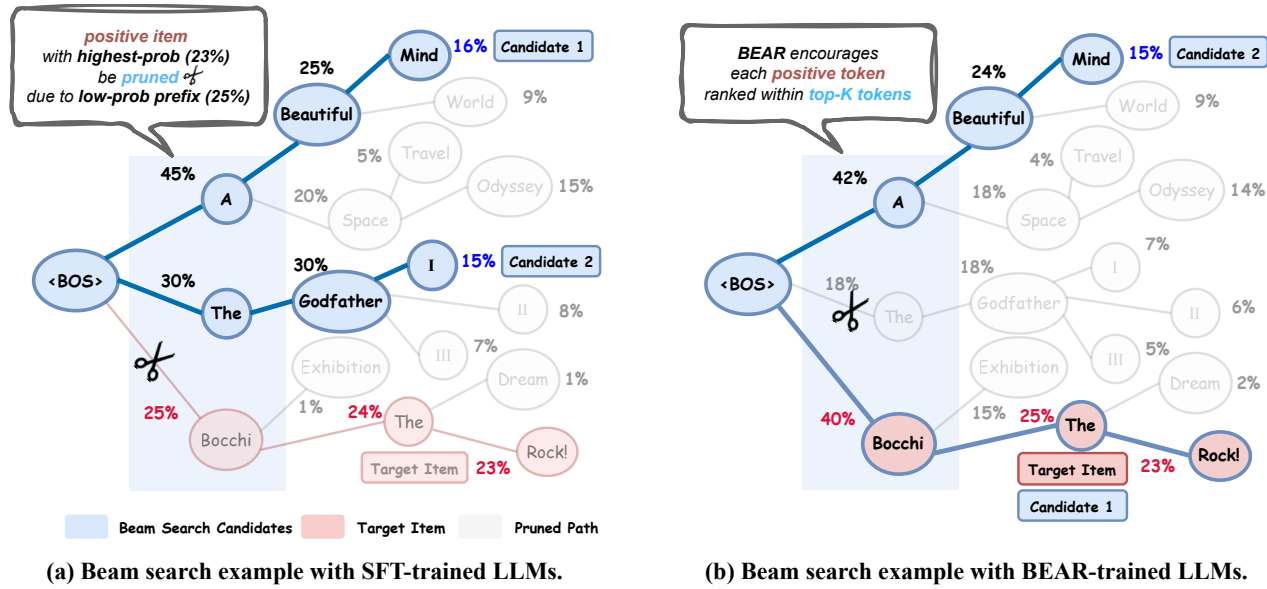


Figure 1: Illustration of beam search applied to SFT-trained LLMs vs. BEAR-trained LLMs (beam width $B = 2$). The **blue paths** represent **candidate items** retrieved by beam search (e.g., "A Beautiful Mind"), while **red circles** mark the **positive item** (e.g., "Bocchi the Rock!"). (a) In the SFT case, although the positive item possesses the highest overall probability, it is pruned at the first step due to the insufficient probability of its prefix (e.g., "Bocchi"). (b) BEAR promotes ranking each positive token within the top- B among all possible tokens, making beam search more likely to retain the positive item.

During inference, beam search is widely adopted to efficiently retrieve items with high probabilities [4, 33, 56]. Beam search incrementally extends sequences token-by-token, retaining only a small number (i.e., the beam width B) of the highest-probability candidates at each step, while discarding the others. This approach is considerably faster than exhaustive retrieval over all items, but it introduces a *greedy pruning* mechanism: if a positive item's prefix has relatively low probability, it may be pruned early, even if its complete sequence has the highest overall item probability.

Challenges. We identify a fundamental *training-inference inconsistency* between SFT optimization and beam search decoding in current LLM-based RS. While SFT targets improving the overall probability of positive items, this objective does not necessarily ensure that the item would be retrieved during beam search. Figure 1(a) illustrates an example: despite the positive item "Bocchi the Rock!" achieving the highest overall probability, its low-probability prefix "Bocchi" causes premature pruning during beam search. Our empirical analysis on real-world datasets reveals that this phenomenon is alarmingly prevalent: over 80% of positive items with top- B overall probability are pruned before reaching the final recommendations of beam search (cf. Figure 3). This striking statistic demonstrates that the training-inference inconsistency constitutes the primary bottleneck in LLM-based RS, demanding urgent resolution.

Our Method. To bridge this gap, we propose a novel fine-tuning objective, termed **BEAR** (Beam-SEarch-Aware Regularization), which explicitly incorporates the beam search dynamics into training. A straightforward yet naive approach would be to simulate beam search for every instance during training, increasing the *prefix*

probabilities of positive items until they rank in the top- B candidate prefixes at each step. However, this approach entails multiple forward passes per instance during training, making it computationally infeasible (cf. Figure 5). Instead, BEAR optimizes a relaxed *necessary condition* for retrieving positive items via beam search, i.e., ensuring each *token* of the positive items ranks within the top- B candidate tokens. Our motivation stems from an interesting empirical observation: violating this necessary condition constitutes the primary cause of incorrect pruning, accounting for over 70% of cases in real-world datasets (cf. Figure 4). Therefore, optimizing BEAR would naturally and substantially mitigate the risk of incorrect pruning, thereby improving recommendation performance (cf. Figure 3). Moreover, unlike the straightforward solution that simulates beam search, BEAR requires no additional forward passes, achieving high computational efficiency with negligible additional overhead compared to standard SFT, making it highly practical for real-world applications (cf. Figure 5).

To empirically validate the effectiveness of BEAR, we conduct comprehensive experiments on four real-world datasets. Experimental results demonstrate that BEAR surpasses nine state-of-the-art fine-tuning baselines for LLM-based RS (mainly published within the past year), with a *notable average improvement of 12.50%*. Moreover, BEAR is a model-agnostic fine-tuning objective and can be seamlessly integrated into various LLM-based RS backbones. Empirically, we evaluate BEAR on three typical recommendation backbones and obtain consistent performance gains. Additional analyses indicate that BEAR can indeed reduce the ratio of incorrect pruning, thereby validating its effectiveness in addressing the training-inference inconsistency.

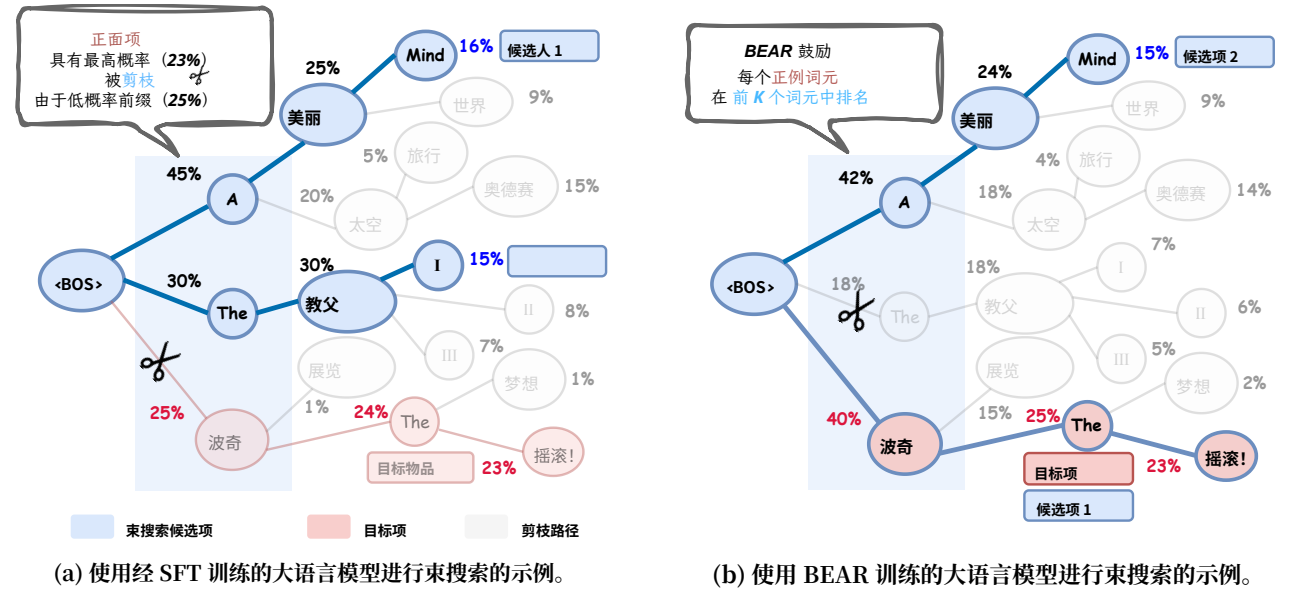


图 1: 束搜索应用于 SFT 训练的 LLM 与 BEAR 训练的 LLM 的示意图 (束宽为 $B = 2$)。其中, **蓝色路径** 表示 **由束搜索检索到的候选项** (例如"A Beautiful Mind"), 而 **红色圆圈** 标记的是 **正样本项** (例如"Bocchi the Rock!"). (a) 在 SFT 情况下, 尽管正样本项具有最高的整体概率, 但由于其前缀 (例如"Bocchi") 的概率不足, 它在第一步就被剪枝了。 (b) BEAR 促使每个正样本令牌在所有可能令牌中排名进入前 B 名, 从而使束搜索更有可能保留正样本项。

在推理阶段, 束搜索 (beam search) 被广泛采用, 以高效检索高概率物品 [4, 33, 56]。束搜索逐令牌增量扩展序列, 在每一步仅保留少量 (即束宽 B) 最高概率的候选项, 同时丢弃其余项。这种方法比穷举检索所有物品快得多, 但引入了一种贪婪剪枝机制: 如果某个正样本物品的前缀概率相对较低, 即使其完整序列具有最高的整体物品概率, 也可能在早期被剪枝。

挑战。 我们识别出一个根本性的训练 - 推理不一致性, 存在于当前基于大语言模型的推荐系统中的监督微调优化与束搜索解码之间。虽然监督微调旨在提升正样本的整体概率, 但该目标并不一定能确保该物品在束搜索过程中被检索到。图1(a)展示了一个示例: 尽管正样本"Bocchi the Rock!" 获得了最高的整体概率, 但其低概率前缀"Bocchi" 导致在束搜索过程中被过早剪枝。我们在真实世界数据集上的实证分析表明, 这一现象令人警觉地普遍存在: 超过 80% 具有顶部 B 整体概率的正样本在到达束搜索的最终推荐之前就被剪枝 (参见图3)。这一惊人的统计数据表明, 训练 - 推理不一致性构成了基于大语言模型的推荐系统的主要瓶颈, 亟需解决。

我们的方法。 为了弥合这一差距, 我们提出了一种新颖的微调目标, 称为 **BEAR** (Beam-SEarch-Aware Regularization, 即束搜索感知正则化), 该方法将束搜索动态显式地纳入训练过程中。一种直接但朴素的方法是在训练期间对每个实例模拟束搜索, 从而增加正项的 前缀

概率, 直到它们在哪一步都位列前- B 个候选前缀之中。然而, 这种方法需要在训练期间对每个实例进行多次前向传播, 导致计算上不可行 (参见图5)。相反, BEAR 优化了一个松弛的 **必要条件**, 用于通过束搜索检索正项, 即确保正项的每个令牌都排在 B 个候选令牌之内。我们的动机源于一个有趣的实证观察: 违反这一必要条件是导致错误剪枝的主要原因, 在真实世界数据集中占比超过 70% (参见图4)。因此, 优化 BEAR 能够自然且显著地降低错误剪枝的风险, 从而提升推荐性能 (参见图3)。此外, 与模拟束搜索的直接方案不同, BEAR 无需额外的前向传播, 在与标准 SFT 相比几乎不增加额外开销的情况下实现了高计算效率, 使其非常适用于实际应用 (参见图5)。

为了实证验证 BEAR 的有效性, 我们在四个真实世界数据集上进行了全面的实验。实验结果表明, BEAR 超越了九种基于大语言模型的推荐系统 (LLM-based RS) 的最先进微调基线 (主要为过去一年发表的方法), 取得了平均 12.50% 的显著提升。此外, BEAR 是一种与模型无关的微调目标, 可以无缝集成到各种基于大语言模型的推荐系统骨干网络中。在实证方面, 我们在三种典型的推荐骨干网络上评估了 BEAR, 并获得了 consistent 的性能提升。进一步的分析表明, BEAR 确实能够降低错误剪枝的比例, 从而验证了其在解决训练 - 推理不一致性问题上的有效性。

Instruction Template	
Instruction:	Given a list of movies the user has watched before, please recommend a new movie that the user likes to the user.
Input:	The user has watched the following movies before: Good Will Hunting , The Man Who Knew Infinity , ... , K-On! , Sound! Euphonium .
Output:	Bocchi the Rock! .

Figure 2: Instruction prompt template.

Contributions. In summary, our contributions are as follows:

- We identify and analyze the critical issue of training-inference inconsistency in LLM-based recommendation tasks, and advocate for explicitly incorporating the impact of beam search into the design of fine-tuning objectives.
- We propose BEAR, a novel beam-search-aware fine-tuning objective that optimizes a necessary condition for retrieving positive items via beam search. This objective effectively mitigates the risk of incorrect pruning in beam search with negligible additional computational overhead.
- We conduct extensive experiments to validate the effectiveness of BEAR, with a notable average improvement of 12.50% over state-of-the-art methods.

2 Preliminaries

In this section, we formulate the task of LLM-based RS in Section 2.1, introduce beam search in Section 2.2, and discuss the limitations of SFT due to the training-inference inconsistency in Section 2.3.

2.1 Task Formulation

In this work, we focus on sequential recommendation [21, 25, 51], a widely adopted recommendation scenario that aims to model the temporal dynamics of user preferences. Formally, given a recommender system with the corresponding user set $\mathcal{U}$ and the item set $\mathcal{I}$, each user $u \in \mathcal{U}$ is associated with a historical interaction sequence $H_u = [h_1, h_2, \dots, h_{N-1}]$, where $h_n \in \mathcal{I}$ is the n -th interacted item. The goal of sequential recommendation is to predict the next item h_N that a user would likely interact with based on the previous interactions H_u .

Recently, given the impressive open-domain knowledge and semantic reasoning capabilities of Large Language Models (LLMs) [1, 2, 52], there has been a surge of interest in leveraging them for sequential recommendation (i.e., LLM-based RS) [3, 4, 26, 32]. In this paradigm, the historical interactions H_u are transformed into a structured prompt x using a predefined template, where each item $h_n \in H_u$ is represented by its textual description (e.g., title or metadata), as illustrated in Figure 2. The prompt x is subsequently employed to instruct LLMs to generate the textual description y of the predicted item, where the generation is driven by the probability $P_\theta(y|x)$ estimated by LLMs with parameters θ .

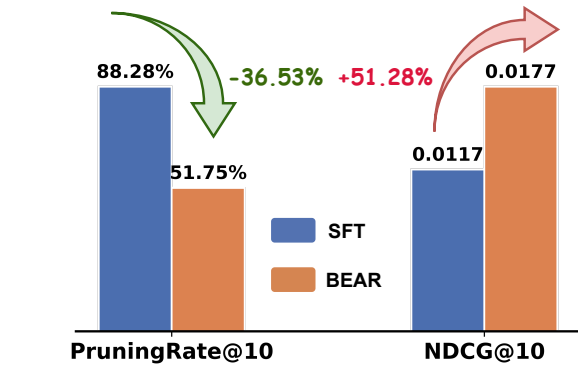


Figure 3: Performance comparison between SFT and BEAR on the Book dataset. PruningRate@10 denotes the pruning rate with beam width $B = 10$, i.e., the proportion of positive items that rank within the top-10 in overall probability yet are pruned during beam search. The complete results are presented in Table 2 and Figure 6.

Notably, LLMs operate at a fine-grained semantic token level, sequentially outputting the tokens of the predicted item y according to the token-wise conditional probability $P_\theta(y_t|y_{<t}, x)$, where y_t denotes the t -th token in y , and $y_{<t}$ denotes the prefix tokens preceding y_t . Consequently, the overall probability of the predicted item y can be factorized as the product of token probabilities, i.e., $P_\theta(y|x) = \prod_{t=1}^{|y|} P_\theta(y_t|y_{<t}, x)$, where $|y|$ is the length of y . Such fine-grained token-level semantic modeling empowers LLMs to capture nuanced user preferences, offering a promising avenue for advancing recommendation performance.

2.2 Beam Search

During the inference stage, LLM-based RS aim to retrieve the top- K items with the highest probabilities $P_\theta(y|x)$. However, given the vast item space and significant computational cost of LLMs, exhaustively enumerating all items and calculating their probabilities is prohibitively expensive. To address this challenge, an efficient approximate decoding strategy, known as *beam search*, is widely adopted in LLM-based RS [4, 33, 56]. This strategy leverages the generative nature of LLMs and incorporates a greedy search principle, achieving a balance between accuracy and efficiency.

Beam Search Procedure. Rather than directly assessing the probabilities $P_\theta(y|x)$ of all items, beam search follows an iterative token-wise extension mechanism. Specifically, at each decoding step, beam search progressively extends the sequences (a.k.a. beams) to their next tokens, and retains only a fixed number (i.e., the beam width B) of the most promising candidate sequences, while pruning the rest. The detailed procedure is as follows (also illustrated in Figure 1):

- *Initialization:* at the initial step $t = 0$, initialize the candidate set $\mathcal{B}_{<1} = \{b_{<1}\}$ with a single candidate $b_{<1} = "<\text{BOS}>"$, where $<\text{BOS}>$ is a special token denoting the beginning of the sequence.
- *Expansion:* at step $t \geq 1$, for each candidate sequence $b_{<t}$ from the previous step's candidate set $\mathcal{B}_{<t}$, extend it with all possible next tokens b_t to form the extended candidates $b_{<t+1}$. The probabilities of the extended candidates are computed as $P_\theta(b_{<t+1}|x) = P_\theta(b_{<t}|x) \cdot P_\theta(b_t|b_{<t}, x)$.

指令模板	
说明:	根据用户之前观看过的电影列表, 请向用户推荐一部他们可能会喜欢的新电影。
输入:	用户之前观看过以下电影: 心灵捕手 , 知无涯者 , ... , 轻音少女! , 吹响! 上低音号 .
输出:	孤独摇滚! .

图 2: 指令提示模板。

贡献。综上所述, 我们的贡献如下:

- 我们识别并分析了基于大语言模型的推荐任务中训练与推理不一致的关键问题, 并主张在微调目标的设计中显式纳入束搜索的影响。
- 我们提出了 BEAR, 一种新颖的束搜索感知微调目标, 该目标优化了通过束搜索检索正样本的必要条件。此目标有效缓解了束搜索中错误剪枝的风险, 且带来的额外计算开销微乎其微。
- 我们进行了大量实验以验证 BEAR 的有效性, 其平均性能较最先进方法提升了显著的 12.50%。

2 预备知识

在本节中, 我们在第2.1节阐述基于大语言模型的推荐系统 (LLM-based RS) 任务, 在第2.2节介绍束搜索算法, 并在第2.3节讨论由于训练与推理不一致导致的监督微调 (SFT) 的局限性。

2.1 任务形式化

在本工作中, 我们专注于序列推荐 [21, 25, 51], 这是一种旨在建模用户偏好时序动态的广泛采用的推荐场景。形式上, 给定一个包含相应用户集合 $\mathcal{U}$ 和物品集合 $\mathcal{I}$ 的推荐系统, 每个用户 $u \in \mathcal{U}$ 都关联一个历史交互序列 $H_u = [h_1, h_2, \dots, h_{N-1}]$, 其中 $h_n \in \mathcal{I}$ 是第 n 个交互物品。序列推荐的目标是基于先前的交互 H_u , 预测用户接下来可能交互的下一个物品 h_N 。

最近, 鉴于大型语言模型 (LLM) 在开放域知识和语义推理能力方面的出色表现 [12, 52], 人们对其在序列推荐中的应用 (即基于 LLM 的推荐系统) 产生了浓厚的兴趣 [3, 4, 26, 32]。在此范式中, 历史交互行为 H_u 通过预定义模板被转化为结构化提示 x , 其中每个项目 $h_n \in H_u$ 均以其文本描述 (例如标题或元数据) 来表示, 如图2所示。该提示 x 随后用于指导 LLM 生成预测项目的文本描述 y , 其中生成过程由概率 $P_\theta(y|x)$ 驱动, 该概率由具有参数 θ 的 LLM 估算得出。

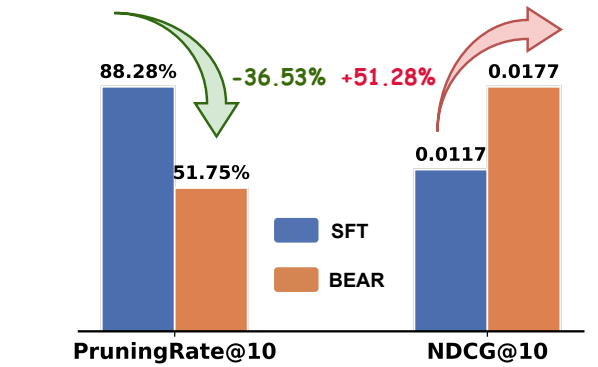


图 3: SFT 与 BEAR 在 Book 数据集上的性能对比。PruningRate@10 表示束宽为 $B = 10$ 时的剪枝率, 即在整体概率中排名前 10 但在束搜索过程中被剪枝的正样本物品比例。完整结果见表2 和图6。

值得注意的是, LLM 在细粒度的语义标记级别上运行, 按顺序输出预测项目的标记 y accord-根据逐词条件概率 $P_\theta(y_t|y_{<t}, x)$, 其中 y_t 表示 y 中的第 t 个令牌, 而 $y_{<t}$ 表示 y_t 之前的前缀令牌。因此, 预测项 y 的整体概率可以分解为令牌概率的乘积, 即 $P_\theta(y|x) = \prod_{t=1}^{|y|} P_\theta(y_t|y_{<t}, x)$, 其中 $|y|$ 是 y 的长度。这种细粒度的令牌级语义建模使大型语言模型能够捕捉细微的用户偏好, 为提升推荐性能提供了一条有前景的途径。

2.2 束搜索

在推理阶段, 基于大语言模型的推荐系统旨在检索概率最高的前 K 个物品 $P_\theta(y|x)$ 。然而, 鉴于庞大的物品空间以及大语言模型显著的计算成本, 穷举所有物品并计算其概率的代价过高。为应对这一挑战, 一种称为束搜索的高效近似解码策略被广泛应用于基于大语言模型的推荐系统 [4, 33, 56]。该策略利用大语言模型的生成特性, 并结合贪心搜索原则, 在准确性与效率之间取得平衡。

束搜索过程。束搜索并非直接评估所有项的概率 $P_\theta(y|x)$, 而是采用一种迭代的逐令牌扩展机制。具体而言, 在每一个解码步骤中, 束搜索会逐步将序列 (即束) 扩展至其下一个令牌, 并仅保留固定数量 (即束宽 B) 的最有希望的候选序列, 同时剪枝其余序列。详细过程如下 (亦见图1):

- *初始化:* 在初始步骤 $t = 0$, 用单个候选项 $b_{<1} = "<\text{BOS}>"$ 初始化候选集 $\mathcal{B}_{<1} = \{b_{<1}\}$, 其中 $<\text{BOS}>$ 是一个表示序列开始的特殊标记。
- *扩展:* 在步骤 $t \geq 1$, 对于上一步候选集 $\mathcal{B}_{<t}$ 中的每个候选序列 $b_{<t}$, 使用所有可能的下一个标记 b_t 对其进行扩展, 以形成扩展后的候选项 $b_{<t+1}$ 。扩展后候选项的概率计算为 $P_\theta(b_{<t+1}|x) = P_\theta(b_{<t}|x) \cdot P_\theta(b_t|b_{<t}, x)$ 。

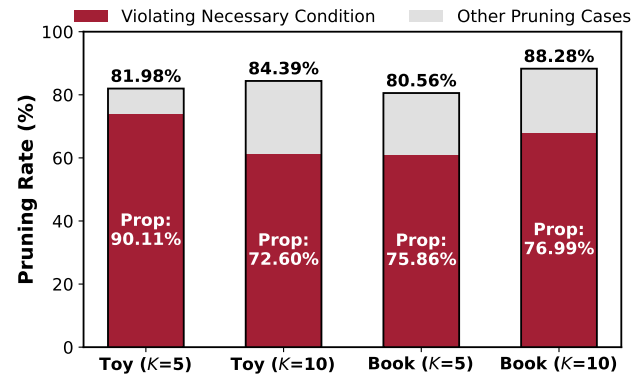


Figure 4: An overwhelming majority of positive items with high overall probabilities (i.e., top- K ranked among candidates) are pruned during beam search, where failure to meet the necessary condition (3.2) is the primary cause (marked as red rectangles). Here, we set beam width $B = 10$ and number of recommended items $K \in \{5, 10\}$. "Prop" denotes the proportion of pruning cases attributable to necessary condition violations (red), consistently accounting for over 70% of all pruning cases (red and gray) across all settings.

- *Pruning*: sort all extended candidates $b_{<t+1}$ by their probabilities $P_\theta(b_{<t+1}|x)$, and retain only the top- B candidates with the highest probabilities as the new candidate set $\mathcal{B}_{<t+1}$ for the next step. All other candidates are pruned.
- *Termination*: repeat the expansion and pruning steps until all candidates are completed by generating a special end-of-sequence token <EOS> or reaching the predefined maximum length. This results in a final candidate set with B completed sequences, from which the top- K items with the highest overall probabilities are selected as the final recommendations.

Suboptimal Pruning in Beam Search. Despite its efficiency, beam search is inherently a locally greedy decoding strategy and is therefore not guaranteed to yield the optimal retrieval results [14]. Specifically, at each decoding step, it retains only the top- B candidates $b_{<t}$ with the highest *prefix probabilities* $P_\theta(b_{<t}|x)$, potentially pruning candidates that could lead to higher *overall probabilities* $P_\theta(y|x)$ after generating subsequent tokens. For example, as illustrated in Figure 1(a), at the first decoding step, the candidate sequence "Bocchi" is pruned because its prefix probability (25%) is lower than that of the other two candidates, i.e., "The" (30%) and "A" (45%). However, one of the complete items starting with this prefix, i.e., "Bocchi the Rock!", indeed has the highest overall probability (23%) among all items. In this case, the positive items with high probability are incorrectly pruned early due to the greedy nature of beam search, which can significantly hinder retrieval accuracy and lead to low-relevance recommendations [55, 70].

2.3 Limitations of SFT

To adapt LLMs to recommendation scenarios, existing LLM-based RS typically employ supervised fine-tuning (SFT) [4, 33, 39, 56]. In this process, the recommendation data is organized into a training dataset consisting of multiple prompt-response pairs (x, y) as

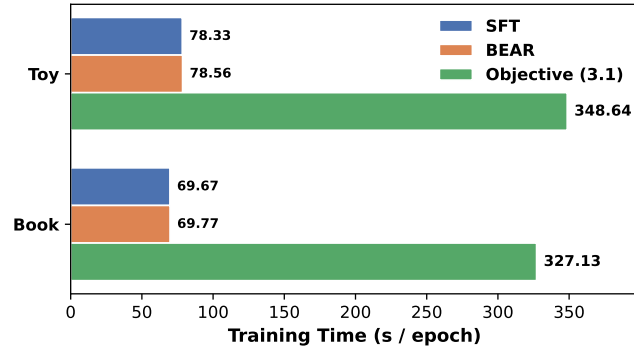


Figure 5: Running time comparison of three training objectives: SFT, BEAR, and objective (3.1). While SFT and BEAR have comparable training time, optimizing the objective (3.1) is significantly more time-consuming due to the need for beam search simulation during training. Refer to Figure 9 for complete results in training efficiency.

training instances, where x is the constructed prompt and y is the textual description of the positive item (cf. Figure 2). Subsequently, the LLM is adapted by optimizing the following SFT objective:

$$\min_{\theta} \mathcal{L}_{\text{SFT}}(x, y; \theta) = -\log P_{\theta}(y|x) = -\sum_{t=1}^{|y|} \log P_{\theta}(y_t|y_{<t}, x). \quad (2.1)$$

By minimizing this loss, the model is guided to maximize the overall probability $P_{\theta}(y|x)$ of the positive item y conditioned on the prompt x , thereby aligning the LLM with recommendation tasks. More importantly, the SFT objective can be decomposed into a sum of token-wise cross-entropy losses (i.e., $-\log P_{\theta}(y_t|y_{<t}, x)$), which can be efficiently optimized through one forward pass of the LLM per training instance, facilitating scalable training.

Training-Inference Inconsistency. While SFT has shown promising results, this conventional strategy could be suboptimal for LLM-based RS due to the *training-inference inconsistency* between SFT learning and beam search decoding. Specifically, the SFT objective is to maximize the overall probability of positive items. However, **during beam search, a high overall probability does not necessarily ensure successful retrieval.** As discussed in Section 2.2, the greedy pruning behavior of beam search can easily eliminate positive items whose prefixes do not exhibit sufficiently high local probability, regardless of their global high probability. As exemplified in Figure 1(a), the positive item "Bocchi the Rock!" is pruned due to the relatively low probability of its prefix "Bocchi", despite having the highest overall probability among all items. Critically, this situation is alarmingly common rather than exceptional. Our empirical analysis, as exhibited in Figures 4 and 6, reveals a striking finding: **among positive items with high overall probabilities, an overwhelming majority (e.g., 88.28% in Book and 84.39% in Toy datasets) are pruned by beam search and consequently fail to appear in the final recommendations.** This remarkable percentage underscores that the training-inference inconsistency represents a fundamental bottleneck in LLM-based RS, severely compromising their effectiveness and demanding immediate attention. Thus, this work explores a novel beam-search-aware fine-tuning approach to mitigate this critical inconsistency.

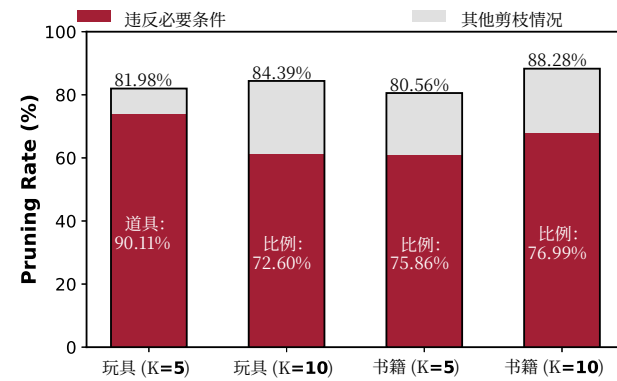


图 4: 绝大多数具有较高总体概率（即在候选项中排名靠前）的正样本在束搜索过程中被剪枝，其中未能满足必要条件（3.2）是主要原因（标记为红色矩形）。此处，我们设置束宽为 $B = 10$ ，推荐项目数量为 $K \in \{5, 10\}$ 。"Prop"表示因违反必要条件而导致的剪枝案例比例（红色），在所有设置下，该比例始终占全部剪枝案例（红色和灰色）的 70% 以上。

- **剪枝**：对所有扩展候选项进行排序 $b_{<t+1}$ 根据它们的概率 $P_{\theta}(b_{<t+1}|x)$ ，并仅保留概率最高的前 B 个候选项作为下一步的新候选集 $\mathcal{B}_{<t+1}$ 。所有其他候选项将被剪枝。
- **终止**：重复扩展和剪枝步骤，直到所有候选项通过生成特殊的序列结束标记 <EOS> 或达到预定义的最大长度而完成。这将产生一个包含 B 个已完成序列的最终候选集，从中选择整体概率最高的前 K 个项目作为最终推荐。

束搜索中的次优剪枝。 尽管束搜索效率很高，但它本质上是一种局部贪心的解码策略，因此无法保证产生最优的检索结果 [14]。具体而言，在每一个解码步骤中，它仅保留具有最高 B 前缀概率 $b_{<t}$ 的前 $b_{<t}|x()$ 个候选项 $P_{\theta}()$ ，这可能会剪除那些在生成后续词元后能够带来更高**整体概率**的候选项 [55, 70]。例如，如图1(a)所示，在第一个解码步骤中，候选序列"Bocchi"因其前缀概率（25%）低于其他两个候选项（即"The"（30%）和"A"（45%））而被剪除。然而，以该前缀开头的完整条目之一，即"Bocchi the Rock!"，实际上在所有条目中具有最高的整体概率（23%）。在这种情况下，由于束搜索的贪心特性，具有高概率的正样本被过早地错误剪除，这会显著阻碍检索准确性并导致低相关性的推荐 [55, 70]。

2.3 SFT 的局限性

为使大语言模型（LLM）适应推荐场景，现有的基于大语言模型的推荐系统通常采用监督微调（SFT）[4, 33, 39, 56]。在此过程中，推荐数据被组织为一个由多个提示-响应配对 (x, y) 组成的训练数据集，

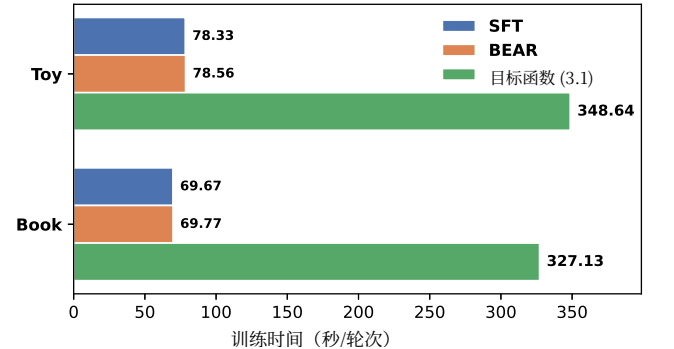


图 5: 三种训练目标的运行时间比较：SFT、BEAR 和目标 (3.1)。虽然 SFT 和 BEAR 的训练时间相当，但优化目标 (3.1) 要耗时得多，因为需要在训练期间进行束搜索模拟。完整训练效率结果请参阅图9。

作为训练实例，其中 x 是构建的提示，而 y 是正样本物品的文本描述（参见图2）。随后，通过优化以下 SFT 目标来适配大语言模型：

$$\min_{\theta} \mathcal{L}_{\text{SFT}}(x, y; \theta) = -\log P_{\theta}(y|x) = -\sum_{t=1}^{|y|} \log P_{\theta}(y_t|y_{<t}, x). \quad (2.1)$$

通过最小化该损失，模型被引导以最大化在提示 x 条件下正样本项 y 的整体概率 $P_{\theta}(y|x)$ ，从而使大语言模型与推荐任务对齐。更重要的是，SFT 目标可以分解为词元级交叉熵损失 (i.e., $-\log P_{\theta}(y_t|y_{<t}, x)$) 之和，这使得每个训练实例只需通过大语言模型的一次前向传播即可高效优化，从而促进了可扩展的训练。

训练 - 推理不一致性。 尽管监督微调（SFT）已展现出令人鼓舞的成果，但由于 *SFT* 学习与束搜索解码之间存在训练 - 推理不一致性，这种传统策略对于基于大语言模型的推荐系统（**LLM-based RS**）而言可能并非最优。具体而言，**SFT** 的目标是最大化正样本的整体概率。然而，在束搜索过程中，较高的整体概率并不能必然保证成功的检索。**正如第2.2节所讨论的**，束搜索的贪婪剪枝行为很容易剔除那些前缀局部概率不够高、但全局概率很高的正样本。如图1（a）所示，正样本《孤独摇滚！》因其前缀“孤独”的概率相对较低而被剪枝，尽管它在所有物品中具有最高的整体概率。关键在于，这种情况绝非特例，而是普遍存在且令人担忧。我们的实证分析（见图4和6）揭示了一个惊人的发现：在那些具有较高整体概率的正样本中，绝大多数（例如在 Book 数据集中为 88.28%，在 Toy 数据集中为 84.39%）都被束搜索剪枝，从而未能出现在最终推荐结果中。这一显著比例凸显了训练 - 推理不一致性构成了基于大语言模型推荐系统的根本性瓶颈，严重损害了其有效性，亟需立即关注。因此，本研究探索了一种新颖的感知束搜索的微调方法，以缓解这一关键的不一致性问题。

3 Methodology

To bridge the training-inference inconsistency, we present **BEAR** (**B**eam-**S**Earch-Aware **R**egularization), a novel *beam-search-aware* objective that explicitly considers the impact of beam search. We first present a straightforward yet time-costly solution in Section 3.1, then introduce our BEAR objective in Section 3.2.

3.1 A Straightforward Solution

In Section 2.3, we have identified the limitations of SFT due to its beam search unawareness. Since SFT merely maximizes the overall probability of the positive item, it cannot prevent its low-probability prefixes from being pruned during beam search inference, which may lead to degraded recommendation performance.

Straightforward Objective. To address this issue, a straightforward idea is to directly optimize the objective for retaining the positive item throughout the beam search process, i.e., ensuring that every prefix $y_{<t}$ of the positive item y is always ranked within the top- B candidates at each beam search step t , where B is the beam width. Formally, this requirement can be expressed as:

$$\max_{\theta} \prod_{t=1}^{|y|+1} \mathbb{I} \left(P_{\theta}(y_{<t}|x) \geq P_{\theta}(b_{<t}^B|x) \right), \quad (3.1)$$

where $b_{<t}^B$ denotes the beam search candidate ranked exactly in the B -th position at decoding step $t - 1$ (cf. Section 2.2), and $\mathbb{I}(\cdot)$ is the indicator function. The right-hand term $P_{\theta}(b_{<t}^B|x)$ can be understood as a threshold — if the probability of the prefix $y_{<t}$, i.e., the left-hand term $P_{\theta}(y_{<t}|x)$, exceeds this threshold $P_{\theta}(b_{<t}^B|x)$, the prefix will be preserved by beam search; otherwise, it will be pruned immediately. Optimizing the objective (3.1) systematically elevates the probability of each positive prefix above the corresponding threshold, thereby reducing the risk that the positive item will be incorrectly pruned during beam search.

Limitations on Computational Efficiency. Despite its conceptual simplicity, this straightforward solution is computationally impractical in real-world settings. Specifically, calculating the threshold $P_{\theta}(b_{<t}^B|x)$ in objective (3.1) necessitates simulating the entire beam search procedure during training for each instance (x, y) . This involves token-by-token expansion of candidate sequences and requires up to B times additional forward passes of the LLM per training instance compared to SFT. Worse still, the strictly sequential dependency makes this process inherently difficult to parallelize and thus heavily time-consuming. Even with a relatively small beam width (e.g., $B = 10$), our empirical analysis shows that optimizing (3.1) incurs over 4.45× runtime cost compared to basic SFT in practice, making it unsuitable for real-world applications (cf. Figure 5). This motivates the development of an efficient surrogate strategy to approximate the objective while avoiding the full beam search simulation during training.

3.2 BEAR: Beam-Search-Aware Regularization

Necessary Condition Objective. While directly optimizing the objective (3.1) is challenging, we empirically observe an intriguing phenomenon: *the majority of incorrect pruning cases are in fact caused by the violation of a necessary condition*, as illustrated in Figure 4. This condition stipulates that the token-wise probability

Table 1: Dataset statistics.

Dataset	#Users	#Items	#Interactions	Density
Office	4895	2414	53149	0.4498%
Book	16559	6344	151928	0.1446%
Toy	19124	11758	165247	0.0735%
Clothing	39230	22948	277534	0.0308%

$P_{\theta}(y_t|y_{<t}, x)$ of each positive token y_t must rank within the top- B among all possible next tokens; otherwise, it will be pruned immediately by the beam search, since only the top- B extended candidates are retained at each decoding step (cf. Section 2.2). Formally, this necessary condition can be expressed as:

$$\max_{\theta} \prod_{t=1}^{|y|} \mathbb{I} \left(P_{\theta}(y_t|y_{<t}, x) \geq \beta_t^B \right), \quad (3.2)$$

where $\beta_t^B = \text{top-}B\{P_{\theta}(\cdot|y_{<t}, x)\}$ denotes the B -th highest token-wise probability among all possible next tokens at step t . In contrast to the original objective (3.1), which enforces that *every positive prefix* is retained within the top- B beam search candidates, this formulation operates at the granularity of *individual positive tokens*. The goal of objective (3.2) is to raise the probability of each positive token $P_{\theta}(y_t|y_{<t}, x)$ above the token-wise threshold β_t^B , thereby ensuring that the extended positive prefix $y_{<t+1}$ at least remains within the top- B among all possible continuations extended from the prefix $y_{<t}$, which is a necessary condition for finally retaining this positive prefix in the next step of beam search.

Notably, although this objective is a *weaker constraint* than (3.1), it remains a strong enough constraint and serves as an *effective and computationally efficient surrogate* for optimization. On the one hand, as shown in Figure 4, failure to satisfy this necessary condition is the *primary* cause of incorrect pruning, accounting for over 70% of all cases. Therefore, directly optimizing this necessary condition can substantially mitigate the training-inference inconsistency and improve recommendation performance. On the other hand, unlike (3.1), this objective does not require simulating the beam search procedure and only involves computing the token-wise top- B threshold β_t^B . Note that the probabilities $P_{\theta}(\cdot|y_{<t}, x)$ for all valid next tokens of each positive prefix $y_{<t}$ can be efficiently calculated by applying a softmax operation to the output logits of LLMs, while these logits are already computed during SFT. Therefore, calculating this top- B threshold β_t^B incurs no additional forward passes of LLMs. As a result, its computational cost is comparable to standard SFT and is significantly more efficient than directly optimizing (3.1), as evidenced by our empirical results in Figures 5 and 9.

Beam-Search-Aware Regularization (BEAR). Despite the potential benefits of optimizing the necessary condition (3.2), it poses significant challenges due to its inherent *discontinuity*, which hinders gradient-based optimization. To tackle these issues, we follow the principle of Empirical Risk Minimization [53] and formulate the risk indicating the violation of this necessary condition as:

$$\mathcal{R}_t(x, y; \theta) = \log \mathbb{I} \left(\log \beta_t^B - \log P_{\theta}(y_t|y_{<t}, x) > 0 \right). \quad (3.3)$$

3 方法论

为了弥合训练与推理之间的不一致性，我们提出了**BEAR**（**B**eam-**S**Earch-Aware **R**egularization，即感知束搜索的正则化），这是一种新颖的感知束搜索的目标函数，能够显式地考虑束搜索的影响。我们在3.1节首先提出一种直接但耗时较长的解决方案，随后在3.2节介绍我们的 BEAR 目标函数。

3.1 一种直接的解决方案

在2.3节中，我们指出了监督微调（SFT）因缺乏束搜索感知而存在的局限性。由于 SFT 仅最大化正样本项的整体概率，它无法防止其低概率前缀在束搜索推理过程中被剪枝，这可能导致推荐性能下降。

直接目标。为解决这一问题，一个直观的思路是直接优化旨在使正样本项在整个束搜索过程中得以保留的目标，即确保正样本项 y 的每个前缀 $y_{<t}$ 在束搜索的每一步 t 中始终排名前- B 位，其中 B 为束宽。形式上，这一要求可表述为：

$$\max_{\theta} \prod_{t=1}^{|y|+1} \mathbb{I} \left(P_{\theta}(y_{<t}|x) \geq P_{\theta}(b_{<t}^B|x) \right), \quad (3.1)$$

其中， $b_{<t}^B$ 表示在解码步骤 $t - 1$ 时恰好排在第 B 位的束搜索候选（参见第2.2节），而 $\mathbb{I}(\cdot)$ 是指示函数。右侧项 $P_{\theta}(b_{<t}^B|x)$ 可理解为一个阈值：如果前缀 $y_{<t}$ 的概率（即左侧项 $P_{\theta}(y_{<t}|x)$ ）超过该阈值 $P_{\theta}(b_{<t}^B|x)$ ，则该前缀将被束搜索保留；否则，它将立即被剪枝。系统地优化目标（3.1）可将每个正样本前缀的概率提升至相应阈值之上，从而降低正样本在束搜索过程中被错误剪枝的风险。

计算效率的局限性。尽管这一直接方案在概念上简单，但在实际场景中却缺乏计算可行性。具体而言，计算目标（3.1）中的阈值 $P_{\theta}(b_{<t}^B|x)$ 需要在训练过程中对每个实例 (x, y) 模拟完整的束搜索过程。这涉及候选序列的逐词扩展，并且与标准监督微调（SFT）相比，每个训练实例最多需要额外进行 B 次大语言模型的前向传播。更糟糕的是，严格的顺序依赖使得该过程本质上难以并行化，因而极其耗时。即使采用相对较小的束宽（例如 $B = 10$ ），我们的实证分析表明，在实践中优化（3.1）所带来的运行时间开销比基础 SFT 高出 4.45× 倍以上，使其不适用于实际应用（参见图5）。这促使我们开发一种高效的代理策略，以近似该目标，同时避免在训练过程中进行完整的束搜索模拟。

3.2 BEAR：束搜索感知正则化

必要条件目标。虽然直接优化目标（3.1）具有挑战性，但我们通过实证观察到一个有趣的现象：大多数错误的剪枝案例实际上是由于违反了必要条件所致，如图4所示。该条件规定，令牌级别的概率

表 1: 数据集统计信息。

数据集	用户数	物品数	交互数	密度
办公	4895	2414	53149	0.4498%
Book	16559	6344	151928	0.1446%
Toy	19124	11758	165247	0.0735%
服装	39230	22948	277534	0.0308%

$P_{\theta}(y_t|y_{<t}, x)$ 对于每个正样本令牌 y_t 必须排名位于前- B 位（在所有可能的下一个令牌中）；否则，它将被束搜索立即剪枝，因为在每个解码步骤中仅保留前- B 个扩展候选项（参见第2.2节）。形式上，这一必要条件可表示为：

$$\max_{\theta} \prod_{t=1}^{|y|} \mathbb{I} \left(P_{\theta}(y_t|y_{<t}, x) \geq \beta_t^B \right), \quad (3.2)$$

其中， $\beta_t^B = \text{top-}B\{P_{\theta}(\cdot|y_{<t}, x)\}$ 表示在步骤 t 时，所有可能的下一个令牌中第 B 高的令牌级概率。与原始目标（3.1）不同，后者要求每个正前缀都必须保留在前 B 个束搜索候选项中，而本公式的操作粒度为单个正令牌。目标（3.2）旨在将每个正令牌 $P_{\theta}(y_t|y_{<t}, x)$ 的概率提升至令牌级阈值 β_t^B 以上，从而确保从前缀 $y_{<t}$ 扩展出的正前缀 $y_{<t+1}$ 至少能位于所有可能续写中的前 B 名，这是该正前缀在束搜索下一步中得以保留的必要条件。

值得注意的是，尽管该目标作为约束比（3.1）更弱，但它仍然是一个足够强的约束，并可作为优化过程中一种有效且计算高效的替代方案。一方面，如图 4 所示，不满足这一必要条件是导致错误剪枝的主要原因，占所有情况的 70% 以上。因此，直接优化这一必要条件可以显著缓解训练与推理之间的一致性，并提升推荐性能。另一方面，与（3.1）不同，该目标无需模拟束搜索过程，仅需计算令牌级的顶部 B 阈值 β_t^B 。请注意，每个正前缀的所有有效下一令牌的概率 $P_{\theta}(\cdot|y_{<t}, x)$ 可以通过对大型语言模型（LLM）的输出 logits 应用 softmax 操作来高效计算，而这些 logits 在监督微调（SFT）过程中已经计算完成。因此，计算该顶部 B 阈值 β_t^B 不会产生额外的大型语言模型前向传播。结果是，其计算成本与标准监督微调相当，且比直接优化（3.1）高效得多，正如我们在图 5 和图 9 中的实证结果所证明的那样。

束搜索感知正则化（BEAR）。尽管优化必要条件（3.2）具有潜在优势，但由于其固有的不连续性，这带来了重大挑战，阻碍了基于梯度的优化。为解决这些问题，我们遵循经验风险最小化原则 [53]，并将表示违反该必要条件的风险表述为：

$$\mathcal{R}_t(x, y; \theta) = \log \mathbb{I} \left(\log \beta_t^B - \log P_{\theta}(y_t|y_{<t}, x) > 0 \right). \quad (3.3)$$

Table 2: Overall performance comparison with various baselines. The LLM-based RS methods are applied ("+") on the BIGRec [3] backbone. N@K and H@K denote NDCG@K and HitRatio@K metrics, respectively. The best results are highlighted in bold, and the best baselines are underlined. "Imp.%" indicates the relative improvement of BEAR over the best baseline.

Methods	Office				Book				Toy				Clothing			
	N@5	N@10	H@5	H@10	N@5	N@10	H@5	H@10	N@5	N@10	H@5	H@10	N@5	N@10	H@5	H@10
SASRec	0.0127	0.0196	0.0223	0.0447	0.0061	0.0080	0.0100	0.0157	0.0089	0.0109	0.0148	0.0211	0.0022	0.0033	0.0036	0.0072
BERT4Rec	0.0164	0.0206	0.0264	0.0396	0.0100	0.0121	<u>0.0158</u>	0.0223	0.0101	0.0118	0.0137	0.0188	0.0028	0.0034	0.0033	0.0053
DROS	0.0195	0.0240	0.0265	0.0405	0.0072	0.0096	<u>0.0116</u>	0.0192	0.0105	0.0126	0.0148	0.0213	0.0038	0.0046	0.0048	0.0074
LLM-CF	0.0161	0.0200	0.0306	0.0426	0.0077	0.0100	0.0141	0.0212	0.0077	0.0101	0.0148	0.0223	0.0022	0.0034	0.0042	0.0078
DLIM2Rec	0.0166	0.0231	0.0322	0.0530	0.0068	0.0094	0.0112	0.0194	0.0080	0.0110	0.0156	0.0248	0.0028	0.0045	0.0056	0.0106
BIGRec	0.0105	0.0226	0.0213	0.0587	0.0068	0.0101	0.0128	0.0228	0.0122	0.0157	0.0200	0.0307	0.0043	0.0071	0.0086	0.0172
+ D ³	0.0142	0.0237	0.0291	0.0592	0.0057	0.0093	0.0112	0.0221	0.0163	0.0202	0.0257	0.0375	0.0045	0.0072	0.0088	0.0172
+ CFT	0.0183	0.0272	0.0306	0.0582	0.0051	0.0090	0.0107	0.0224	0.0138	0.0173	0.0217	0.0325	0.0045	0.0076	0.0086	0.0182
+ D ² LR	0.0243	0.0311	0.0395	0.0603	0.0070	0.0089	0.0128	0.0183	0.0142	0.0177	0.0227	0.0338	0.0051	0.0081	0.0098	0.0194
+ IGD	0.0187	0.0266	0.0312	0.0566	0.0061	0.0095	0.0125	0.0228	0.0191	0.0239	0.0273	0.0421	0.0059	0.0084	0.0098	0.0178
+ MSL	<u>0.0346</u>	<u>0.0388</u>	<u>0.0504</u>	0.0634	0.0087	0.0117	0.0137	0.0228	<u>0.0239</u>	<u>0.0301</u>	<u>0.0359</u>	<u>0.0551</u>	<u>0.0075</u>	<u>0.0115</u>	0.0136	0.0260
+ S-DPO	0.0209	0.0292	0.0374	0.0629	0.0082	0.0102	0.0125	0.0189	0.0181	0.0227	0.0288	0.0432	0.0054	0.0093	0.0110	0.0232
+ RosePO	0.0266	0.0338	0.0436	0.0660	<u>0.0117</u>	<u>0.0127</u>	0.0153	0.0185	0.0114	0.0174	0.0194	0.0378	0.0066	0.0110	<u>0.0138</u>	0.0272
+ SPRec	0.0249	0.0326	0.0416	0.0655	0.0072	0.0110	0.0130	<u>0.0244</u>	0.0148	0.0175	0.0234	0.0317	0.0047	0.0069	0.0082	0.0152
+ BEAR	0.0365	0.0421	0.0545	0.0717	0.0146	0.0177	0.0203	0.0297	0.0254	0.0316	0.0380	0.0574	0.0082	0.0127	0.0148	0.0290
Imp.%	+5.49%	+8.51%	+8.13%	+8.64%	+24.79%	+39.37%	+28.48%	+21.72%	+6.28%	+4.98%	+5.85%	+4.17%	+9.33%	+10.43%	+7.25%	+6.62%

Here, the $\Delta_t^B = \log \beta_t^B - \log P_\theta(y_t|y_{<t}, x)$ term¹ can be interpreted as the *pruning margin* at step t . A positive pruning margin indicates that the token-wise probability of y_t falls below the top- B threshold β_t^B , which corresponds to the violation of the necessary condition (3.2) and leads to immediate pruning at step t . Therefore, minimizing this risk for each step is fundamentally equivalent to optimizing the necessary condition for beam search survival.

To enable gradient-based optimization and effectively minimize the risk (3.3), we surrogate the indicator function $\mathbb{I}(\Delta_t^B > 0)$ with the sigmoid function $\sigma_\xi(\Delta_t^B) = 1/(1 + \exp(-\Delta_t^B/\xi))$, where ξ is a temperature hyperparameter that controls the smoothness² [66, 67]. This surrogate provides a smooth and tight approximation to the discontinuous indicator function, resulting in the following differentiable *beam-search-aware regularization* term³:

$$\mathcal{L}_{\text{reg}}(x, y; \theta) = \sum_{t=1}^{|y|} \log \sigma_\xi \left(\log \beta_t^B - \log P_\theta(y_t|y_{<t}, x) \right). \quad (3.4)$$

By minimizing this regularization term, the model is guided to minimize the pruning margin Δ_t^B at each step, thereby effectively reducing the risk of incorrect pruning during beam search. Notably, while the standard SFT objective (2.1) also increases the probabilities of positive tokens, it treats all tokens equally without considering the beam search dynamics, leading to massive premature pruning and noteworthy training-inference inconsistency (cf. Section 2.3). In contrast, regularization (3.4) explicitly takes the pruning margin into account, dynamically evaluating the probabilities of positive tokens $P_\theta(y_t|y_{<t}, x)$ based on their different top- B thresholds β_t^B , thus effectively mitigating the training-inference inconsistency.

¹In objective (3.2), the indicator function $\mathbb{I}(P_\theta(y_t|y_{<t}, x) \geq \beta_t^B)$ is equivalent to $\mathbb{I}(\log \beta_t^B - \log P_\theta(y_t|y_{<t}, x) \leq 0)$, which operates on the difference of log-probabilities and thus facilitates optimization.

²As the temperature $\xi \rightarrow 0$, the sigmoid function $\sigma_\xi(\Delta_t^B)$ converges to $\mathbb{I}(\Delta_t^B > 0)$.

³In objectives (3.3) and (3.4), we take the logarithm to convert maximization of the product into minimization of the sum of risks, facilitating token-wise optimization. For mathematically rigorously, we assume that the token probabilities are lower bounded by a small positive constant, ensuring the well-definedness of the logarithm operation [62, 66, 67].

Finally, by incorporating the regularization term (3.4) into the SFT objective (2.1), we obtain the proposed **BEAR objective**:

$$\min_{\theta} \mathcal{L}_{\text{BEAR}}(x, y; \theta) = \mathcal{L}_{\text{SFT}}(x, y; \theta) + \lambda \mathcal{L}_{\text{reg}}(x, y; \theta), \quad (3.5)$$

where $\lambda > 0$ is a weight hyperparameter that controls the regularization strength. By incorporating beam-search awareness with SFT, BEAR not only improves the overall probability of positive items (cf. Table 2), but also effectively reduces the risk of incorrectly pruning positive items during beam search (cf. Figure 6). These effects collectively alleviate the training-inference inconsistency and enhance recommendation performance, leading to an effective and efficient fine-tuning objective for LLM-based RS.

4 Experiments

In this section, we conduct extensive experiments to evaluate the effectiveness of the proposed BEAR objective. Our aim is to answer the following research questions (RQs):

- **RQ1:** How does BEAR perform in recommendation tasks compared to existing methods?
- **RQ2:** How does BEAR mitigate the training-inference inconsistency and alleviate the incorrect pruning during beam search?
- **RQ3:** How does BEAR generalize to different LLM-based RS backbones, LLM sizes, and fine-tuning objectives?
- **RQ4:** How does the computational efficiency of BEAR compare to other methods?
- **RQ5:** How do the hyperparameters λ and ξ of BEAR affect its performance?

4.1 Experimental Setup

Datasets. To ensure a fair comparison with existing methods, we conduct experiments on four widely used Amazon datasets [20, 40], including Office, Book, Toy, and Clothing. The dataset preprocessing conforms to the standard procedure in previous studies [3, 56], where the users and items with fewer than 5 interactions are filtered out (i.e., 5-core), and a sliding window of size 11 is applied to segment the user interaction sequences. The resulting sequences

表 2：与各种基线的整体性能比较。基于大语言模型的推荐系统方法应用于 ("+") BIGRec [3]骨干网络。N@K 和 H@K 分别表示 NDCG@K 和 HitRatio@K 指标。最佳结果以粗体突出显示，最佳基线以下划线标示。"提升%"表示 BEAR 相对于最佳基线的相对提升幅度。

方法	办公用品				Book				Toy				服装			
	N@5	N@10	H@5	H@10	N@5	N@10	H@5	H@10	N@5	N@10	H@5	H@10	N@5	N@10	H@5	H@10
SASRec	0.0127	0.0196	0.0223	0.0447	0.0061	0.0080	0.0100	0.0157	0.0089	0.0109	0.0148	0.0211	0.0022	0.0033	0.0036	0.0072
BERT4Rec	0.0164	0.0206	0.0264	0.0396	0.0100	0.0121	<u>0.0158</u>	0.0223	0.0101	0.0118	0.0137	0.0188	0.0028	0.0034	0.0033	0.0053
DROS	0.0195	0.0240	0.0265	0.0405	0.0072	0.0096	<u>0.0116</u>	0.0192	0.0105	0.0126	0.0148	0.0213	0.0038	0.0046	0.0048	0.0074
LLM-CF	0.0161	0.0200	0.0306	0.0426	0.0077	0.0100	0.0141	0.0212	0.0077	0.0101	0.0148	0.0223	0.0022	0.0034	0.0042	0.0078
DLIM2Rec	0.0166	0.0231	0.0322	0.0530	0.0068	0.0094	0.0112	0.0194	0.0080	0.0110	0.0156	0.0248	0.0028	0.0045	0.0056	0.0106
BIGRec	0.0105	0.0226	0.0213	0.0587	0.0068	0.0101	0.0128	0.0228	0.0122	0.0157	0.0200	0.0307	0.0043	0.0071	0.0086	0.0172
+ D ³	0.0142	0.0237	0.0291	0.0592	0.0057	0.0093	0.0112	0.0221	0.0163	0.0202	0.0257	0.0375	0.0045	0.0072	0.0088	0.0172
+ CFT	0.0183	0.0272	0.0306	0.0582	0.0051	0.0090	0.0107	0.0224	0.0138	0.0173	0.0217	0.0325	0.0045	0.0076	0.0086	0.0182
+ D ² LR	0.0243	0.0311	0.0395	0.0603	0.0070	0.0089	0.0128	0.0183	0.0142	0.0177	0.0227	0.0338	0.0051	0.0081	0.0098	0.0194
+ IGD	0.0187	0.0266	0.0312	0.0566	0.0061	0.0095	0.0125	0.0228	0.0191	0.0239	0.0273	0.0421	0.0059	0.0084	0.0098	0.0178
+ MSL	<u>0.0346</u>	<u>0.0388</u>	<u>0.0504</u>	0.0634	0.0087	0.0117	0.0137	0.0228	<u>0.0239</u>	<u>0.0301</u>	<u>0.0359</u>	<u>0.0551</u>	<u>0.0075</u>	<u>0.0115</u>	0.0136	0.0260
+ S-DPO	0.0209	0.0292	0.0374	0.0629	0.0082	0.0102	0.0125	0.0189	0.0181	0.0227	0.0288	0.0432	0.0054	0.0093	0.0110	0.0232
+ RosePO	0.0266	0.0338	0.0436	0.0660	<u>0.0117</u>	<u>0.0127</u>	0.0153	0.0185	0.0114	0.0174	0.0194	0.0378	0.0066	0.0110	<u>0.0138</u>	0.0272
+ SPRec	0.0249	0.0326	0.0416	0.0655	0.0072	0.0110	0.0130	<u>0.0244</u>	0.0148	0.0175	0.0234	0.0317	0.0047	0.0069	0.0082	0.0152
+ 买入印象%	0.0365	0.0421	0.0545	0.0717	0.0146	0.0177	0.0203	0.0297	0.0254	0.0316	0.0380	0.0574	0.0082	0.0127	0.0148	0.0290
	+5.49%	+8.51%	+8.13%	+8.64%	+24.79%	+39.37%	+28.48%	+21.72%	+6.28%	+4.98%	+5.85%	+4.17%	+9.33%	+10.43%	+7.25%	+6.62%

在此， $\Delta_t^B = \log \beta_t^B - \log P_\theta(y_t|y_{<t}, x)$ 项¹可解释为步骤 t 处的剪枝边界。正的剪枝边界表明 y_t 的令牌级概率低于前 B 阈值 β_t^B ，这对应于违反必要条件 (3.2)，并导致在步骤 t 立即剪枝。因此，最小化每一步的这种风险，本质上等同于优化束搜索生存的必要条件。

为了启用基于梯度的优化并有效最小化风险 (3.3)，我们用 Sigmoid 函数 $\sigma_\xi(\Delta_t^B) = 1/(1 + \exp(-\Delta_t^B/\xi))$ 替代指示函数 $\mathbb{I}(\Delta_t^B > 0)$ ，其中 ξ 是控制平滑度的温度超参数² [66, 67]。该代

理函数为不连续的指示函数提供了平滑且紧密的近似，从而得到以下可微的束搜索感知正则化项³:

$$\mathcal{L}_{\text{reg}}(x, y; \theta) = \sum_{t=1}^{|y|} \log \sigma_\xi \left(\log \beta_t^B - \log P_\theta(y_t|y_{<t}, x) \right). \quad (3.4)$$

通过最小化该正则化项，模型被引导以在每一步最小化剪枝边界 Δ_t^B ，从而有效降低束搜索过程中错误剪枝的风险。值得注意的是，虽然标准的监督微调 (SFT) 目标 (2.1) 也能提高正样本令牌的概率，但其对所有令牌一视同仁，未考虑束搜索的动态特性，导致大量过早剪枝以及显著的训练 - 推理不一致性 (参见第 2.3 节)。相比之下，正则化项 (3.4) 明确将剪枝边界纳入考量，根据正样本令牌不同的前 $P_\theta(y_t|y_{<t}, x)$ 个 k 个 B 阈值 β_t^B 动态评估其概率，从而有效缓解训练与推理之间的不一致性。

¹在目标函数 (3.2) 中，指示函数 $\mathbb{I}(P_\theta(y_t|y_{<t}, x) \geq \beta_t^B)$ 等价于 $\mathbb{I}(\log \beta_t^B - \log P_\theta(y_t|y_{<t}, x) \leq 0)$ ，其作用于对数概率之差，从而便于优化。²当温度 $\xi \rightarrow 0$ 时，Sigmoid 函数 $\sigma_\xi(\Delta_t^B)$ 收敛于 $\mathbb{I}(\Delta_t^B > 0)$ 。³在目标函数 (3.3) 和 (3.4) 中，我们取对数以将乘积的最大化转化为风险和的最小化，从而便于逐词元优化。为确保数学严谨性，我们假设词元概率有下界 (为一个小的正常数)，以保证对数运算的良好定义性 [62, 66, 67]。

最后，通过将正则化项 (3.4) 融入 SFT 目标 (2.1)，我们得到了所提出的 **BEAR 目标**:

$$\min_{\theta} \mathcal{L}_{\text{BEAR}}(x, y; \theta) = \mathcal{L}_{\text{SFT}}(x, y; \theta) + \lambda \mathcal{L}_{\text{reg}}(x, y; \theta), \quad (3.5)$$

其中 $\lambda > 0$ 是一个权重超参数，用于控制正则化强度。通过将束搜索感知与监督微调 (SFT) 相结合，BEAR 不仅提高了正样本的整体概率 (参见表 2)，还有效降低了在束搜索过程中错误剪枝正样本的风险 (参见图 6)。这些效应共同缓解了训练与推理之间的不一致性，提升了推荐性能，从而为基于大语言模型的推荐系统提供了一种高效且有效的微调目标。

4 实验

在本节中，我们开展了大量实验以评估所提出的 BEAR 目标的有效性。我们的目标是回答以下研究问题 (RQs)：

- **RQ1:** 与现有方法相比，BEAR 在推荐任务中的表现如何？

- **RQ2:** BEAR 如何缓解训练与推理之间的不一致性，并减轻束搜索过程中的错误剪枝问题？
- **RQ3:** BEAR 如何泛化到不同的基于 LLM 的推荐系统骨干网络、LLM 规模以及微调目标？
- **RQ4:** BEAR 的计算效率与其他方法相比如何？

- **RQ5:** BEAR 的超参数 λ 和 ξ 如何影响其性能？

4.1 实验设置

数据集。为了确保与现有方法进行公平比较，我们在四个广泛使用的亚马逊数据集 [20, 40]，(包括 Office、Book、Toy 和 Clothing) 上进行了实验。数据集预处理遵循先前研究中的标准流程 [3, 56]，即过滤掉交互次数少于 5 次的用户和物品 (即 5-core)，并应用大小为 11 的滑动窗口对用户交互序列进行分割。所得到的序列

Table 3: Performance comparison using different recommendation backbones, including LLaRA [32] and A-LLMRec [26]. We adopt MSL [56] as the representative SFT method. $N@K$ and $H@K$ denote $NDCG@K$ and $HitRatio@K$ metrics, respectively. The best results are highlighted in bold. "Imp.%" indicates the relative improvement of BEAR over the best baseline.

Methods	Office				Book				Toy				Clothing			
	N@5	N@10	H@5	H@10	N@5	N@10	H@5	H@10	N@5	N@10	H@5	H@10	N@5	N@10	H@5	H@10
LLaRA	0.0110	0.0234	0.0234	0.0618	0.0068	0.0113	0.0135	0.0267	0.0129	0.0163	0.0204	0.0309	0.0036	0.0071	0.0074	0.0184
+ MSL	0.0380	0.0423	0.0561	0.0696	0.0078	0.0105	0.0132	0.0217	0.0230	0.0290	0.0346	0.0534	0.0082	0.0116	0.0146	0.0254
+ BEAR	0.0407	0.0457	0.0592	0.0748	0.0123	0.0157	0.0212	0.0315	0.0259	0.0315	0.0386	0.0559	0.0089	0.0129	0.0156	0.0284
Imp.%	+7.11%	+8.04%	+5.53%	+7.47%	+57.69%	+38.94%	+57.04%	+17.98%	+12.61%	+8.62%	+11.56%	+4.68%	+8.54%	+11.21%	+6.85%	+11.81%
A-LLMRec	0.0125	0.0247	0.0265	0.0639	0.0077	0.0111	0.0153	0.0256	0.0126	0.0160	0.0198	0.0305	0.0036	0.0068	0.0072	0.0172
+ MSL	0.0332	0.0373	0.0525	0.0655	0.0094	0.0118	0.0151	0.0224	0.0247	0.0302	0.0361	0.0532	0.0081	0.0124	0.0140	0.0276
+ BEAR	0.0392	0.0444	0.0561	0.0722	0.0106	0.0149	0.0189	0.0317	0.0255	0.0312	0.0373	0.0549	0.0087	0.0128	0.0164	0.0288
Imp.%	+18.07%	+19.03%	+6.86%	+10.23%	+12.77%	+26.27%	+23.53%	+23.83%	+3.24%	+3.31%	+3.32%	+3.20%	+7.41%	+3.23%	+17.14%	+4.35%

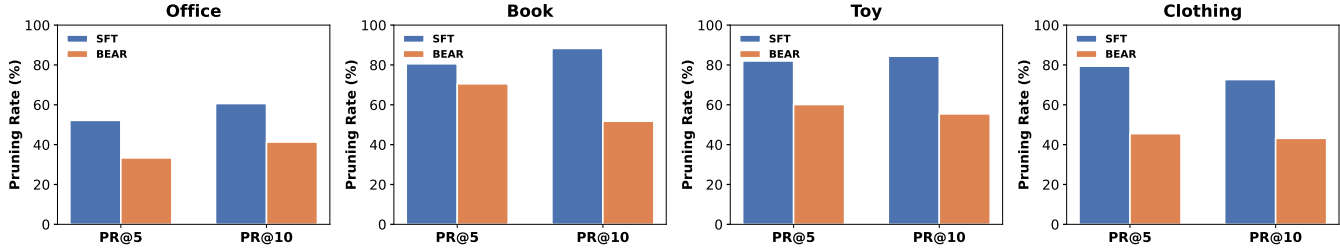


Figure 6: Pruning Rate (PR@K) comparison between SFT and BEAR. During beam search with beam size B , $PR@K$ measures the proportion of incorrectly pruned positive items with high overall probabilities (i.e., top- K ranked among candidates). A lower $PR@K$ indicates better training-inference alignment. Results show that BEAR significantly reduces $PR@K$ compared to SFT.

are then split into training, validation, and test sets with a ratio of 8:1:1 based on the chronological order. The dataset statistics are summarized in Table 1.

Compared Methods. We compare BEAR with the following representative methods:

- **Conventional RS:** traditional methods for sequential recommendation, including SASRec (ICDM ’18) [25], BERT4Rec (CIKM ’19) [48], and DROS (SIGIR ’23) [68].
- **LLM-enhanced RS:** conventional RS models augmented with LLMs as semantic encoder or knowledge enhancer, including LLM-CF (CIKM ’24) [49] and DLLM2Rec (RecSys ’24) [11].
- **LLM-based RS Backbones:** LLMs directly employed for recommendation. Typical backbones include BIGRec (TORS ’25) [3], LLaRA (SIGIR ’24) [32], and A-LLMRec (KDD ’24) [26].
- **SFT Methods for LLM-based RS:** various SFT optimization methods applied on LLM-based RS, including D³ (EMNLP ’24) [4], CFT (CoRR ’24) [73], D²LR (SIGIR ’25) [39], IGD (CoRR ’25) [33], and the state-of-the-art method MSL (SIGIR ’25) [56].
- **DPO Methods for LLM-based RS:** direct preference optimization (DPO) [45] methods that model pairwise preferences for LLM-based RS, including S-DPO (NeurIPS ’24) [10], RosePO (CoRR ’24) [31], and SPRec (WWW ’25) [15].

Implementation Details. For conventional sequential recommendation methods, we closely follow their official implementations and default hyperparameters, training a 2-layer and 64-dim transformer for 200 epochs with a batch size of 128 and a learning rate of 1e-3. For LLM-enhanced RS methods, we directly utilize their official code, where SASRec is adopted as the backbone. For LLM-based RS methods, Llama-3.2-3B [19] is employed as the recommendation base model in the main experiments (we also evaluate the performance

of BEAR on other LLM sizes in Table 4, including Llama-3.2-1B and Llama-3-8B [19]). Following the recent works [56], we fine-tune the LLMs for 10 epochs and evaluate the checkpoints with the best NDCG@10 results on the validation set. During training, the standard LoRA technique [24] and AdamW [38] optimizer are applied, with a rank of 8 and a learning rate of 1e-4 (except for SPRec, which uses 2e-5). The batch size is set to 64 for DPO methods, and 128 for others. During inference, following previous works [3, 4, 56], we use constrained beam search with a beam size B of 10, ensuring that the generated items are valid and non-hallucinated. For evaluation, we adopt the widely-used NDCG@ K and HitRatio@ K metrics to assess the top- K recommendation performance, with $K \in \{5, 10\}$. For our BEAR method, we search hyperparameters $\lambda \in [0.1, 5.0]$ and $\xi \in [0.25, 3.0]$. For all the baseline SFT and DPO-based fine-tuning methods, we closely follow their official implementations and recommended hyperparameters. For hardware, all experiments are conducted on 8× GeForce RTX 5090 GPUs.

4.2 Experimental Results

In the following, we present and analyze the experimental results to answer the aforementioned research questions.

Overall Performance (RQ1). Table 2 presents the overall recommendation performance comparison of BEAR with baselines. As revealed, BEAR consistently outperforms all compared methods across all datasets, yielding *an average improvement of +12.50%* in NDCG@ K and HitRatio@ K over the state-of-the-art SFT methods (e.g., MSL) and DPO methods (e.g., RosePO). This demonstrates the effectiveness of BEAR in enhancing recommendation quality, which aligns well with our motivation of bridging the training-inference gap in LLM-based RS (cf. Section 3.2).

表 3: 使用不同推荐骨干网络（包括 LLaRA）的性能对比 [32] 和 A-LLMRec [26]。我们采用 MSL [56] 作为代表性的 SFT 方法。 $N@K$ 和 $H@K$ 分别表示 $NDCG@K$ 和 $HitRatio@K$ 指标。最佳结果以粗体突出显示。"Imp.%"表示 BEAR 相对于最佳基线的相对提升幅度。

方法	办公用品				Book				Toy				服装			
	N@5	N@10	H@5	H@10	N@5	N@10	H@5	H@10	N@5	N@10	H@5	H@10	N@5	N@10	H@5	H@10
LLaRA	0.0110	0.0234	0.0234	0.0618	0.0068	0.0113	0.0135	0.0267	0.0129	0.0163	0.0204	0.0309	0.0036	0.0071	0.0074	0.0184
+ MSL	0.0380	0.0423	0.0561	0.0696	0.0078	0.0105	0.0132	0.0217	0.0230	0.0290	0.0346	0.0534	0.0082	0.0116	0.0146	0.0254
+ BEAR	0.0407	0.0457	0.0592	0.0748	0.0123	0.0157	0.0212	0.0315	0.0259	0.0315	0.0386	0.0559	0.0089	0.0129	0.0156	0.0284
提升幅度%	+7.11%	+8.04%	+5.53%	+7.47%	+57.69%	+38.94%	+57.04%	+17.98%	+12.61%	+8.62%	+11.56%	+4.68%	+8.54%	+11.21%	+6.85%	+11.81%
A-LLMRec	0.0125	0.0247	0.0265	0.0639	0.0077	0.0111	0.0153	0.0256	0.0126	0.0160	0.0198	0.0305	0.0036	0.0068	0.0072	0.0172
+ MSL	0.0332	0.0373	0.0525	0.0655	0.0094	0.0118	0.0151	0.0224	0.0247	0.0302	0.0361	0.0532	0.0081	0.0124	0.0140	0.0276
+ BEAR	0.0392	0.0444	0.0561	0.0722	0.0106	0.0149	0.0189	0.0317	0.0255	0.0312	0.0373	0.0549	0.0087	0.0128	0.0164	0.0288
提升幅度/%	+18.07%	+19.03%	+6.86%	+10.23%	+12.77%	+26.27%	+23.53%	+23.83%	+3.24%	+3.31%	+3.32%	+3.20%	+7.41%	+3.23%	+17.14%	+4.35%

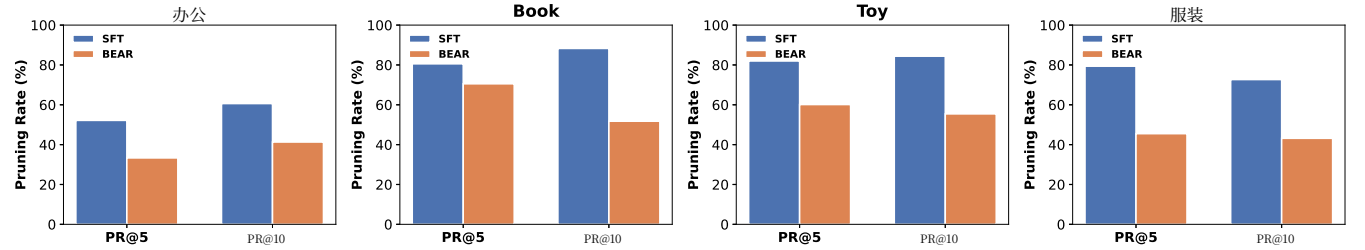


图 6: SFT 与 BEAR 之间的剪枝率 (PR@K) 对比。在束搜索（束大小为 B ）期间， $PR@K$ 衡量的是具有高整体概率（即在候选项中排名前 K ）的正样本被错误剪枝的比例。较低的 $PR@K$ 表示更好的训练 - 推理对齐。结果表明，与 SFT 相比，BEAR 显著降低了 $PR@K$ 。

随后按时间顺序以 8:1:1 的比例划分为训练集、验证集和测试集。数据集统计信息总结于表1中。

对比方法。 我们将 BEAR 与以下代表性方法进行比较：

- **传统推荐系统：** 传统的序列推荐方法，包括 SASRec (ICDM’18) [25]、BERT4Rec (CIKM ’19) [48], 和 DROS (SIGIR ’23) [68]。
- **大语言模型增强型推荐系统：** 将大语言模型作为语义编码器或知识增强器融入的传统推荐系统模型，包括 LLM-CF (CIKM ’24) [49] 和 DLLM2Rec (RecSys ’24) [11]。
- **基于大语言模型的推荐系统骨干网络：** 直接采用大语言模型进行推荐。典型的骨干网络包括 BIGRec (TORS ’25) [3]、LLaRA (SIGIR ’24) [32] 和 A-LLMRec (KDD ’24) [26]。
- **基于大语言模型推荐系统的监督微调方法：** 应用于基于大语言模型推荐系统的各种监督微调优化方法，包括 D³ (EMNLP ’24) [4], CFT (CoRR ’24) [73]、D²LR (SIGIR ’25) [39]、IGD (CoRR ’25) [33], 以及最先进的方法 MSL (SIGIR ’25) [56]。
- **基于大语言模型的推荐系统的 DPO 方法：** 直接偏好优化 (DPO) [45] 方法，用于对基于大语言模型的推荐系统中的成对偏好进行建模，包括 S-DPO (NeurIPS ’24) [10], RosePO (CoRR ’24) [31], 和 SPRec (WWW ’25) [15]。

实现细节。 对于传统的序列推荐方法，我们严格遵循其官方实现和默认超参数，训练一个 2 层、64 维的 Transformer 模型，共 200 个轮次，批量大小为 128，学习率为 1e-3。对于基于大语言模型增强的推荐系统方法，我们直接使用其官方代码，其中采用 SASRec 作为骨干网络。对于基于大语言模型的推荐系统方法，在主要实验中采用 Llama-3.2-3B [19] 作为推荐基础模型（我们也在表中评估了性能

BEAR 在其他大语言模型规模上的表现，见表4，包括 Llama-3.2-1B 和 Llama-3-8B [19]）。遵循近期工作 [56], , 我们对大语言模型进行 10 个轮次的微调，并在验证集上评估具有最佳 NDCG@10 结果的检查点。训练期间，应用标准的 LoRA 技术 [24] 和 AdamW [38] 优化器，秩设为 8，学习率为 1e-4（SPRec 除外，其使用 2e-5）。DPO 方法的批量大小设为 64，其他方法则为 128。推理阶段，遵循先前工作 [3, 4, 56], , 我们使用约束束搜索，束大小 B 为 10，以确保生成的物品有效且无幻觉。评估方面，我们采用广泛使用的 NDCG@ K 和 HitRatio@ K 指标来评估前- K 推荐性能，其中 $K \in \{5, 10\}$ 。对于我们的 BEAR 方法，我们搜索超参数 $\lambda \in [0.1, 5.0]$ 和 $\xi \in [0.25, 3.0]$ 。对于所有基线的 SFT 和基于 DPO 的微调方法，我们严格遵循其官方实现和推荐的超参数。硬件方面，所有实验均在 8× GeForce RTX 5090 GPU 上进行。

4.2 实验结果

下文将展示并分析实验结果，以回答上述研究问题。

整体性能 (RQ1)。 表2 展示了 BEAR 与基线方法的整体推荐性能对比。结果显示，BEAR 在所有数据集上均持续优于所有对比方法，相较于最先进的 SFT 方法（如 MSL）和 DPO 方法（如 RosePO），其 NDCG@和 HitRatio@平均提升了 +12.50%。这证明了 BEAR 在提升推荐质量方面的有效性，也与我们弥合基于大语言模型的推荐系统中训练与推理差距的初衷相一致（参见第3.2节）。

Table 4: Performance comparison on different LLMs, including Llama-3.2-1B and Llama-3-8B [19], on the BIGRec [3] backbone. We adopt MSL [56] as the representative SFT method. N@K and H@K denote NDCG@K and HitRatio@K metrics, respectively. The best results are highlighted in bold. "Imp.%" indicates the relative improvement of BEAR over the best baseline.

Methods	Office				Book				Toy				Clothing			
	N@5	N@10	H@5	H@10	N@5	N@10	H@5	H@10	N@5	N@10	H@5	H@10	N@5	N@10	H@5	H@10
BIGRec (1B)	0.0095	0.0177	0.0203	0.0447	0.0058	0.0066	0.0114	0.0137	0.0088	0.0133	0.0159	0.0296	0.0032	0.0066	0.0068	0.0176
+ MSL	0.0201	0.0232	0.0322	0.0421	0.0070	0.0089	0.0116	0.0176	0.0165	0.0230	0.0267	0.0465	0.0071	0.0105	0.0136	0.0244
+ BEAR	0.0203	0.0270	0.0343	0.0556	0.0076	0.0096	0.0132	0.0194	0.0176	0.0240	0.0284	0.0484	0.0075	0.0116	0.0144	0.0270
Imp.%	+1.00%	+16.38%	+6.52%	+24.38%	+8.57%	+7.87%	+13.79%	+10.23%	+6.67%	+4.35%	+6.37%	+4.09%	+5.63%	+10.48%	+5.88%	+10.66%
BIGRec (8B)	0.0114	0.0213	0.0213	0.0514	0.0109	0.0137	0.0169	0.0258	0.0142	0.0189	0.0213	0.0357	0.0057	0.0081	0.0104	0.0180
+ MSL	0.0392	0.0417	0.0556	0.0634	0.0103	0.0135	0.0160	0.0260	0.0245	0.0298	0.0373	0.0538	0.0085	0.0128	0.0156	0.0292
+ BEAR	0.0407	0.0438	0.0608	0.0706	0.0140	0.0172	0.0256	0.0356	0.0262	0.0318	0.0401	0.0572	0.0090	0.0135	0.0162	0.0302
Imp.%	+3.83%	+5.04%	+9.35%	+11.36%	+28.44%	+25.55%	+51.48%	+36.92%	+6.94%	+6.71%	+7.51%	+6.32%	+5.88%	+5.47%	+3.85%	+3.42%

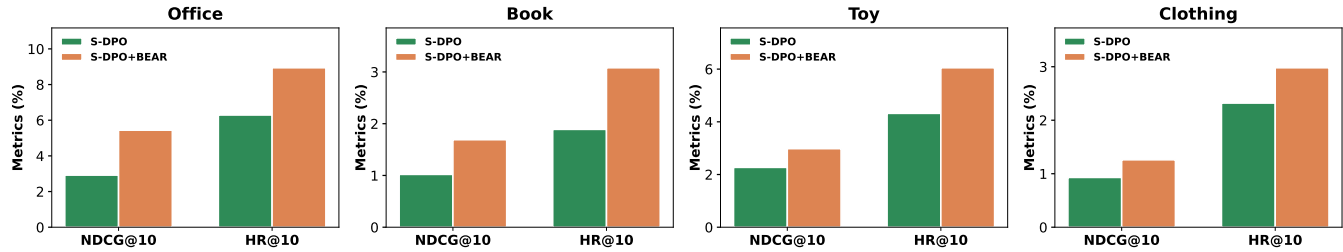


Figure 7: Performance of applying BEAR to S-DPO [10]. In addition to the original S-DPO loss, we apply the positive BEAR regularization to the positive items and the negative BEAR regularization to the sampled negative items. Results show that BEAR significantly enhances the performance of S-DPO across all datasets.

Mitigation of Training-Inference Inconsistency (RQ2). To validate the effectiveness of BEAR in alleviating the incorrect pruning of positive items during beam search, we compare the pruning rate metrics (i.e., PR@K in Figure 6) between BEAR and SFT methods (e.g., MSL). Specifically, PR@K measures the proportion of positive items that are ranked within the top-K based on their overall probabilities but are incorrectly pruned during beam search. As shown in Figure 6, BEAR exhibits a significantly lower PR@K, achieving *an average reduction of -24.86%* compared to SFT methods. This verifies that our beam-search-aware regularization (3.4) can effectively reduce the risk of incorrectly pruning positive items, thereby mitigating the training-inference inconsistency in LLM-based RS.

Generalizability: Backbones and LLM Sizes (RQ3-1). To assess the generalizability of BEAR, we further evaluate its performance on different LLM-based RS backbones (e.g., LLaRA [32] and A-LLMRec [26]) with different base LLM sizes (e.g., Llama-3.2-1B and Llama-3-8B [19]). As illustrated in Table 3 and Table 4, BEAR consistently outperforms the SFT methods across different backbones and LLM sizes, demonstrating its versatility and effectiveness.

Generalizability: Fine-tuning Objectives (RQ3-2). Given that the training-inference inconsistency is a common challenge in fine-tuning LLMs for recommendation, exploring the applicability of BEAR to other fine-tuning objectives, e.g., DPO-based methods, is also of great interest. Therefore, we also apply BEAR to the widely-adopted S-DPO [10]. Specifically, in addition to the original S-DPO loss, we apply the *positive* BEAR regularization in (3.4) (i.e., $\mathcal{L}_{\text{reg}}$) to the positive items, while applying the *negative* regularization (i.e., $-\mathcal{L}_{\text{reg}}$) to the sampled negative items. This encourages the model to retain positive items during beam search and prune negative items, thus aligning the DPO training with the inference

process. As shown in Figure 7, BEAR significantly enhances the performance of S-DPO, validating its broad applicability in improving various fine-tuning objectives for LLM-based RS.

Computational Efficiency (RQ4). As illustrated in Figure 9, BEAR demonstrates both comparable training efficiency to SFT methods and optimal recommendation performance among all compared methods. This should be attributed to BEAR’s meticulous design, which avoids additional forward passes of the LLM during training, as discussed in Section 3.2. In contrast, other methods such as CFT [73], S-DPO [10], and SPRec [15] require additional forward passes for each training instance, leading to noteworthy increased computational overhead (typically 2× to 10× compared to BEAR). This inefficiency hinders their scalability in real-world applications, making BEAR a more practical choice for deploying LLM-based recommender systems.

Hyperparameter Sensitivity (RQ5). In Figure 8, we conduct sensitivity analysis on the hyperparameters λ and ξ of BEAR. As illustrated, the performance for both λ and ξ exhibits a clear peak. Specifically, the regularization weight λ serves to balance the SFT and the regularization: when λ is too small, BEAR degenerates to standard SFT, leading to suboptimal performance due to the training-inference inconsistency; conversely, an excessively large λ overemphasizes the regularization, potentially failing to maximize the overall probability of positive items. For the temperature parameter ξ , it controls the smoothness of the approximation to the indicator function: when ξ is too small, the approximation becomes overly sharp, resulting in gradient saturation; on the other hand, a too-large ξ leads to an overly smooth approximation, diluting

表 4: 不同大语言模型（包括 Llama-3.2-1B 和 Llama-3-8B）的性能对比 [19],在 BIGRec [3]骨干网络上。我们采用 MSL [56] 作为代表性的监督微调（SFT）方法。N@K 和 H@K 分别表示 NDCG@K 和命中率（HitRatio@）K 指标。最佳结果以粗体突出显示。"提升百分比（Imp.%）"表示 BEAR 相对于最佳基线的相对提升。

方法	办公室				Book				Toy				服装			
	N@5	N@10	H@5	H@10	N@5	N@10	H@5	H@10	N@5	N@10	H@5	H@10	N@5	N@10	H@5	H@10
BIGRec (1B)	0.0095	0.0177	0.0203	0.0447	0.0058	0.0066	0.0114	0.0137	0.0088	0.0133	0.0159	0.0296	0.0032	0.0066	0.0068	0.0176
+ MSL	0.0201	0.0232	0.0322	0.0421	0.0070	0.0089	0.0116	0.0176	0.0165	0.0230	0.0267	0.0465	0.0071	0.0105	0.0136	0.0244
+ BEAR	0.0203	0.0270	0.0343	0.0556	0.0076	0.0096	0.0132	0.0194	0.0176	0.0240	0.0284	0.0484	0.0075	0.0116	0.0144	0.0270
提升幅度%	+1.00%	+16.38%	+6.52%	+24.38%	+8.57%	+7.87%	+13.79%	+10.23%	+6.67%	+4.35%	+6.37%	+4.09%	+5.63%	+10.48%	+5.88%	+10.66%
BIGRec (8B)	0.0114	0.0213	0.0213	0.0514	0.0109	0.0137	0.0169	0.0258	0.0142	0.0189	0.0213	0.0357	0.0057	0.0081	0.0104	0.0180
+ MSL	0.0392	0.0417	0.0556	0.0634	0.0103	0.0135	0.0160	0.0260	0.0245	0.0298	0.0373	0.0538	0.0085	0.0128	0.0156	0.0292
+ BEAR	0.0407	0.0438	0.0608	0.0706	0.0140	0.0172	0.0256	0.0356	0.0262	0.0318	0.0401	0.0572	0.0090	0.0135	0.0162	0.0302
提升幅度%	+3.83%	+5.04%	+9.35%	+11.36%	+28.44%	+25.55%	+51.48%	+36.92%	+6.94%	+6.71%	+7.51%	+6.32%	+5.88%	+5.47%	+3.85%	+3.42%

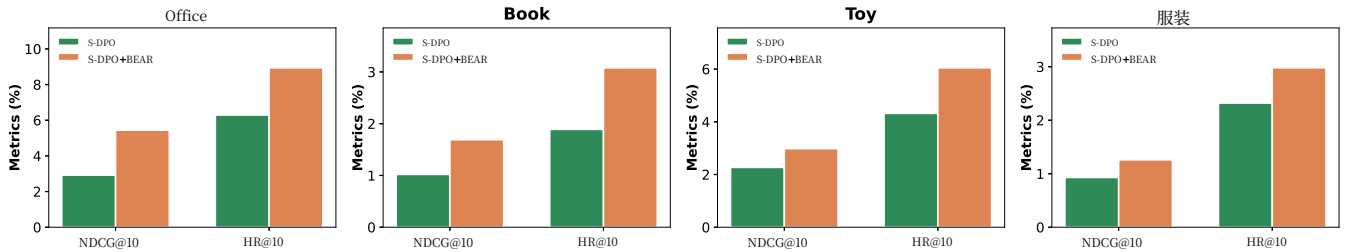


图 7: 将 BEAR 应用于 S-DPO 的性能 [10]。除了原始的 S-DPO 损失外，我们还对正样本项应用了正向 BEAR 正则化，对采样的负样本项应用了负向 BEAR 正则化。结果表明，BEAR 在所有数据集上均显著提升了 S-DPO 的性能。

训练 - 推理不一致性的缓解（RQ2）。为验证 BEAR 在缓解束搜索过程中正样本被错误剪枝方面的有效性，我们对比了 BEAR 与 SFT 方法（如 MSL）的剪枝率指标（即图6中的 PR@）。具体而言，PR@衡量的是那些根据总体概率排名位于前K、却在束搜索过程中被错误剪枝的正样本比例。如图6所示，BEAR 表现出显著更低的 PR@，相较于 SFT 方法平均降低了 *24.86%*。这证实了我们的束搜索感知正则化方法（3.4）能够有效降低正样本被错误剪枝的风险，从而缓解基于大语言模型的推荐系统中的训练 - 推理不一致性问题。

泛化性：骨干网络与 LLM 规模（RQ3-1）。为了评估 BEAR 的泛化能力，我们进一步在不同的基于 LLM 的推荐系统骨干网络（例如 LLaRA [32] 和 A-LLMRec [26]）以及不同的基础 LLM 规模（例如 Llama-3.2-1B 和 Llama-3-8B [19]）上评估了其性能。如表3 和表4所示，BEAR 在不同的骨干网络和 LLM 规模下始终优于 SFT 方法，证明了其多功能性和有效性。

泛化性：微调目标（研究问题 3-2）。鉴于训练与推理不一致是大语言模型推荐系统微调中的常见挑战，探索 BEAR 在其他微调目标（例如基于 DPO 的方法）上的适用性也具有重要意义。因此，我们将 BEAR 应用于广泛采用的 S-DPO [10]。具体而言，除了原始的 S-DPO 损失外，我们将正样本 BEAR 正则化项（见公式3.4），即 $\mathcal{L}_{\text{reg}}$ 应用于正样本，同时将负样本正则化项（即 $-\mathcal{L}_{\text{reg}}$ ）应用于采样的负样本。这促使模型在束搜索过程中保留正样本并剪枝负样本，从而使 DPO 训练与推理过程保持一致。

过程。如图7所示，BEAR 显著提升了 S-DPO 的性能，验证了其在改进各类大语言模型推荐系统微调目标方面的广泛适用性。

计算效率（RQ4）。如图9所示，BEAR 展现出与 SFT 方法相当的训练效率，并在所有对比方法中实现了最优的推荐性能。这归功于 BEAR 的精心设计，正如第3.2节所述，该设计避免了训练期间 LLM 的额外前向传播。相比之下，其他方法如 CFT [73], S-DPO [10], 和 SPRec [15] 需要对每个训练实例进行额外的前向传播，导致计算开销显著增加（通常比 BEAR 高出 2× 至 10×）。这种低效性阻碍了它们在实际应用中的可扩展性，使得 BEAR 成为部署基于 LLM 的推荐系统更实用的选择。

超参数敏感性分析（研究问题 5）。在图8中，我们对 BEAR 的超参数 λ 和 ξ 进行了敏感性分析。如图所示， λ 和 ξ 的性能均呈现出明显的峰值。具体而言，正则化权重 λ 用于平衡监督微调（SFT）与正则化项：当 λ 过小时，BEAR 退化为标准的 SFT，由于训练与推理不一致而导致性能次优；相反，若 λ 过大，则会过度强调正则化，可能导致无法最大化正样本的整体概率。对于温度参数 ξ ，它控制着对指示函数逼近的平滑程度：当 ξ 过小时，逼近变得过于尖锐，导致梯度饱和；另一方面，若 ξ 过大，则会导致逼近过于平滑，从而稀释

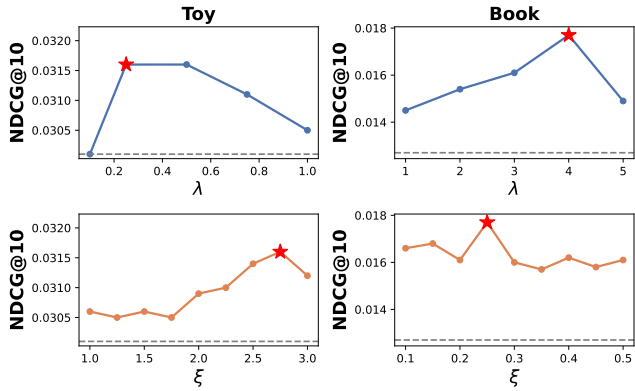


Figure 8: Sensitivity analysis of BEAR’s hyperparameters λ (blue) and ξ (orange). The best result is highlighted with a star, and the baseline result is indicated with a dashed line.

the regularization signal. In addition, we observe that BEAR is relatively robust to hyperparameter variations within a reasonable range, which is beneficial for practical applications.

5 Related Work

Sequential Recommendation. Sequential recommendation [8, 9, 28, 57] has long been a central task in recommendation research, aiming to predict a user’s next interaction based on historical behavior sequences. Classic models such as GRU4Rec [21], Caser [51], SASRec [25], and BERT4Rec [48] have achieved remarkable success by leveraging different architectures to model temporal dependencies [22, 27, 54]. In recent years, by scaling up model sizes to billions of parameters [71], integrating multi-side information [61, 64, 74], performing contextual reasoning [34, 50], and enabling continual learning [7, 41], modern sequential recommendation models have significantly advanced the capabilities of recommender systems.

LLM-based Recommender Systems. Large Language Models (LLMs) [1, 2, 52] have shown exceptional generative [6], reasoning [60], and generalization [44] abilities, inspiring research on their integration into recommender systems (i.e., LLM-based RS) [13, 16, 23, 35, 36, 58]. In this paradigm, recommendation is typically reformulated as a natural language task, where the LLMs are adapted to generate the textual descriptions of the recommended items based on the user history and instructions, thereby leveraging their powerful capabilities for recommendation [5, 12, 17, 26, 30, 32, 47, 59, 72].

Recently, a series of optimization methods have been proposed to enhance the performance of LLM-based RS. Following the standard practice in NLP, supervised fine-tuning (SFT) [3, 4, 42] is widely adopted, aiming to maximize the positive item likelihood. Various enhancements to the basic SFT have been proposed, focusing on causality [73], popularity [39], entropy [33], and masking [56]. Built upon SFT, some researchers have also explored variants of direct preference optimization (DPO) [45], which contrasts preferred and rejected items to model pairwise preferences, including strategies like multiple negatives ranking [10], negative sampling strategies [31], and self-play mechanism [15].

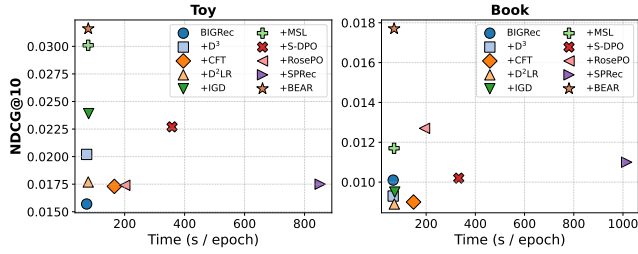


Figure 9: Training efficiency comparison between BEAR and other baselines. The x-axis represents the training time per epoch, and the y-axis represents the NDCG@10 performance.

Despite these promising results, existing fine-tuning strategies still face significant limitations due to the ignorance of the training-inference inconsistency (cf. Section 2.3). Specifically, they typically optimize the overall probability of positive items, which does not guarantee that these items will be retained during the greedy pruning of beam search. This discrepancy can lead to suboptimal recommendation performance, as the final beam search candidates may not include the most relevant items. Targeting this issue, our proposed BEAR method tries to bridge this inconsistency by explicitly considering the beam search process during training, thereby enhancing the recommendation performance of LLM-based RS.

Related Work on Generative Retrieval. Recent studies in other domains, such as generative retrieval [29, 63], have explored the similar training-inference inconsistency issue caused by beam search. These works can be broadly categorized into three main approaches. The first approach, e.g., OTM [76] and DTR [37], focuses on learning a tree model that is Bayes optimal for beam search, or equivalently, satisfies the max-heap property (i.e., each parent node’s preference should be the maximum among its children). The Bayes optimality ensures that the items with top- B overall probabilities will never be pruned. However, transferring this idea to LLM-based RS is non-trivial, as training such a tree model typically requires performing beam search during training, which is computationally impractical for LLMs, as discussed in Section 3.1. The second approach, e.g., RIPOR [69], involves optimizing the prefix probabilities of positive items against negatives to reduce incorrect pruning, which is similar to the objective (3.1). Unfortunately, these methods rely on a large amount of negative sampling, incurring over a 100× computational overhead, rendering it impractical for LLM-based RS. The third approach, e.g., ProphetNet-Ads [43] and PAG [70], heuristically incorporates future token or prior item scores to guide beam search. Nonetheless, accurately estimating these look-ahead scores is equivalent to learning a precise recommender, which is inherently difficult. Different from these strategies, our BEAR tackles this issue from a more principled perspective by optimizing a necessary condition for retrieving positive items during beam search, achieving better performance and computational efficiency.

6 Conclusion and Future Directions

In this paper, we identify and address the training-inference inconsistency in LLM-based recommender systems, which arises from the discrepancy between the SFT training objective and the greedy pruning nature of beam search. To bridge this gap, we propose

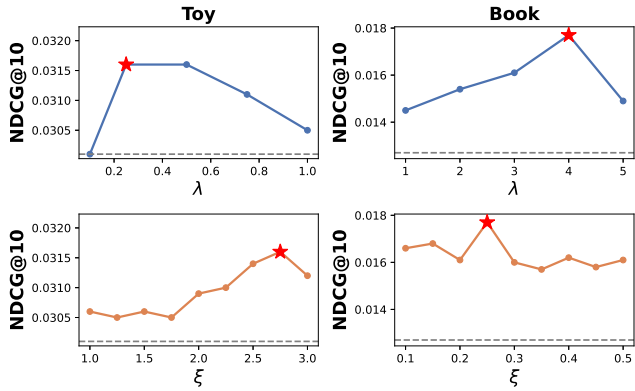


图 8: BEAR 超参数的敏感性分析 λ (蓝色) 和 ξ (橙色)。最佳结果以星号标出, 基线结果以虚线表示。

正则化信号。此外, 我们观察到 BEAR 在合理范围内对超参数变化具有相对的鲁棒性, 这有利于实际应用。

5 相关工作

序列推荐。序列推荐 [8, 9, 28, 57] 长期以来一直是推荐研究中的核心任务, 旨在根据用户的历史行为序列预测其下一次交互。经典模型如 GRU4Rec [21], Caser [51], SASRec [25], 和 BERT4Rec [48] 通过利用不同的架构建模时间依赖关系, 取得了显著成功 [22, 27, 54]。近年来, 通过将模型规模扩展至数十亿参数 [71], 整合多源信息 [61, 64, 74], 执行上下文推理 [34, 50], 并实现持续学习 [7, 41], 现代序列推荐模型显著提升了推荐系统的能力。

基于大语言模型的推荐系统。大语言模型 (LLM) [1, 2, 52] 展现了卓越的生成式 [6], 推理 [60], 与泛化 [44] 能力, 激发了将其融入推荐系统 (即基于大语言模型的推荐系统) 的研究 [13, 16, 23, 35, 36, 58]。

在此范式下, 推荐任务通常被重新表述为自然语言任务, 即让大语言模型根据用户历史和指令生成推荐物品的文本描述, 从而利用其强大的推荐能力 [5, 12, 17, 26, 30, 32, 47, 59, 72]。

最近, 一系列优化方法被提出以提升基于大语言模型的推荐系统 (LLM-based RS) 的性能。遵循自然语言处理 (NLP) 领域的标准实践, 监督微调 (SFT) [3, 4, 42] 被广泛采用, 旨在最大化正样本物品的似然概率。针对基础 SFT 的各种改进方案相继问世, 重点关注因果性 [73]、流行度 [39]、熵 [33], 以及掩码机制 [56]。在 SFT 的基础上, 部分研究者还探索了直接偏好优化 (DPO) [45], 的变体, 该方法通过对比优选物品与拒绝物品来建模成对偏好, 相关策略包括多负样本排序 [10]、负采样策略 [31], 以及自博弈机制 [15]。

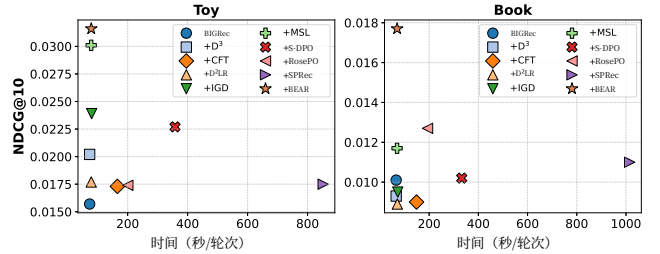


图 9: BEAR 与其他基线方法的训练效率对比。横轴表示每个轮次的训练时间, 纵轴表示 NDCG@10 性能。

尽管取得了这些令人鼓舞的成果, 现有的微调策略仍面临显著局限, 其原因在于忽视了训练与推理之间的不一致性 (参见 2.3 节)。具体而言, 这些方法通常优化正样本物品的整体概率, 但这并不能保证这些物品在束搜索的贪婪剪枝过程中被保留。这种差异可能导致推荐性能次优, 因为最终的束搜索候选集可能未包含最相关的物品。针对这一问题, 我们提出的 BEAR 方法试图通过在训练阶段显式考虑束搜索过程来弥合这种不一致性, 从而提升基于大语言模型的推荐系统的性能。

关于生成式检索的相关工作。其他领域 (如生成式检索) 的最新研究 [29, 63], 已探讨了由束搜索引起的类似训练 - 推理不一致问题。这些工作大致可分为三类主要方法。第一类方法 (例如 OTM [76] 和 DTR [37],) 专注于学习一种对束搜索而言贝叶斯最优的树模型, 或者等价地说, 满足最大堆性质 (即每个父节点的偏好值应为其子节点中的最大值)。贝叶斯最优性确保了具有最高整体概率的物品永远不会被剪枝。然而, 将这一思想迁移到基于大语言模型的推荐系统中并非易事, 因为训练此类树模型通常需要在训练过程中执行束搜索, 正如第 3.1 节所述, 这对大语言模型而言在计算上是不可行的。第二类方法 (例如 RIPOR [69],) 涉及针对负样本优化正样本的前缀概率, 以减少错误剪枝, 这与目标 (3.1) 相似。不幸的是, 这些方法依赖大量的负采样, 导致超过 100× 的计算开销, 使其不适用于基于大语言模型的推荐系统。第三类方法 (例如 ProphetNet-Ads [43] 和 PAG [70],) 启发式地融合未来令牌或先验物品得分来指导束搜索。尽管如此, 准确估计这些前瞻得分等同于学习一个精确的推荐器, 这本质上非常困难。与这些策略不同, 我们的 BEAR 从更原则性的角度解决这一问题, 通过优化束搜索期间检索正样本的必要条件, 实现了更好的性能和计算效率。

6 结论与未来方向

在本文中, 我们识别并解决了基于大语言模型的推荐系统中存在的训练 - 推理不一致问题, 该问题源于监督微调 (SFT) 训练目标与束搜索的贪婪剪枝特性之间的差异。为弥合这一差距, 我们提出

Table 5: Performance comparison between SFT, BEAR, and the straightforward solution that directly optimizes the objective (3.1), on the BIGRec [3] backbone. Due to the high computational cost of beam search required for optimizing (3.1), experiments are conducted on two small-scale datasets (Toy-S and Book-S) with 2K sampled users.

Methods	Toy-S		Book-S	
	NDCG@10	HR@10	NDCG@10	HR@10
+ SFT	0.0076	0.0200	0.0046	0.0120
+ BEAR	0.0094	0.0260	0.0072	0.0180
+ Obj. (3.1)	0.0103	0.0280	0.0077	0.0200

BEAR, a novel beam-search-aware regularization method that explicitly considers the beam search behavior during training. BEAR optimizes a necessary condition for retaining positive items during beam search, thereby reducing the risk of incorrect pruning and enhancing recommendation performance, while incurring minimal computational overhead compared to standard SFT. For future directions, exploring the integration of BEAR with inference-stage techniques, such as multi-token prediction [18] and diverse beam search [55], would be a promising avenue.

A Appendix: Further Discussions

BEAR vs. Straightforward Solution. As discussed in Section 3.1, a naive solution to alleviate the incorrect pruning issue is directly optimizing the objective (3.1). While this straightforward approach seems "theoretically optimal", it requires performing beam search during training, incurring impractical computational overhead in LLM-based RS. Instead, our proposed BEAR method opts to optimize a necessary condition for retaining positive items, offering a more computationally efficient alternative. A crucial research question naturally arises: *Does optimizing this weaker, necessary condition sacrifice significant effectiveness compared to the "optimal" yet expensive solution?*

To answer this, we conduct experiments comparing BEAR with directly optimizing (3.1) on two datasets in Table 5. As observed, both BEAR and the computationally intensive objective (3.1) significantly outperform standard SFT, validating their ability to mitigate training-inference inconsistency. More importantly, the performance gap between BEAR and objective (3.1) is marginal. This indicates that BEAR is *sufficiently effective* in capturing the core dynamics of beam search pruning, achieving performance parity with the straightforward solution while avoiding its severe computational costs. Notably, these results also align perfectly with our finding in Figure 4: since violating the necessary condition accounts for the vast majority of incorrect pruning cases, effectively targeting this primary cause via BEAR yields the majority of the potential gains, which further explains its strong performance.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Floren-
cia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal
Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*
(2023).
- [2] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan,
Wenbin Ge, Yu Han, Fei Huang, et al. 2023. Qwen technical report. *arXiv preprint
arXiv:2309.16609* (2023).

- [3] Kebin Bao, Jizhi Zhang, Wenjie Wang, Yang Zhang, Zhengyi Yang, Yanchen
Luo, Chong Chen, Fuli Feng, and Qi Tian. 2025. A bi-step grounding paradigm
for large language models in recommendation systems. *ACM Transactions on
Recommender Systems* 3, 4 (2025), 1–27.
- [4] Kebin Bao, Jizhi Zhang, Yang Zhang, Xinyue Huo, Chong Chen, and Fuli Feng.
2024. Decoding matters: Addressing amplification bias and homogeneity issue in
recommendations for large language models. In *Proceedings of the 2024 Conference
on Empirical Methods in Natural Language Processing*. 10540–10552.
- [5] Kebin Bao, Jizhi Zhang, Yang Zhang, Wenjie Wang, Fuli Feng, and Xiangnan
He. 2023. Tallrec: An effective and efficient tuning framework to align large
language model with recommendation. In *Proceedings of the 17th ACM conference
on recommender systems*. 1007–1014.
- [6] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan,
Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda
Askell, et al. 2020. Language models are few-shot learners. *Advances in neural
information processing systems* 33 (2020), 1877–1901.
- [7] Guohao Cai, Jieming Zhu, Quanyu Dai, Zhenhua Dong, Xiuqiang He, Ruiming
Tang, and Rui Zhang. 2022. Reloop: A self-correction continual learning loop
for recommender systems. In *Proceedings of the 45th International ACM SIGIR
Conference on Research and Development in Information Retrieval*. 2692–2697.
- [8] Jianxin Chang, Chen Gao, Yu Zheng, Yiqun Hui, Yanan Niu, Yang Song, Depeng
Jin, and Yong Li. 2021. Sequential recommendation with graph neural networks.
In *Proceedings of the 44th international ACM SIGIR conference on research and
development in information retrieval*. 378–387.
- [9] Xu Chen, Hongteng Xu, Yongfeng Zhang, Jiayi Tang, Yixin Cao, Zheng Qin, and
Hongyuan Zha. 2018. Sequential recommendation with user memory networks.
In *Proceedings of the eleventh ACM international conference on web search and
data mining*. 108–116.
- [10] Yuxin Chen, Junfei Tan, An Zhang, Zhengyi Yang, Leheng Sheng, Enzhi Zhang,
Xiang Wang, and Tat-Seng Chua. 2024. On softmax direct preference optimization
for recommendation. *Advances in Neural Information Processing Systems* 37 (2024),
27463–27489.
- [11] Yu Cui, Feng Liu, Pengbo Wang, Bohao Wang, Heng Tang, Yi Wan, Jun Wang, and
Jiawei Chen. 2024. Distillation matters: empowering sequential recommenders
to match the performance of large language models. In *Proceedings of the 18th
ACM Conference on Recommender Systems*. 507–517.
- [12] Zeyu Cui, Jianxin Ma, Chang Zhou, Jingren Zhou, and Hongxia Yang. 2022.
M6-rec: Generative pretrained language models are open-ended recommender
systems. *arXiv preprint arXiv:2205.08084* (2022).
- [13] Sunhao Dai, Ninglu Shao, Haiyuan Zhao, Weijie Yu, Zihua Si, Chen Xu, Zhongx-
iang Sun, Xiao Zhang, and Jun Xu. 2023. Uncovering chatgpt’s capabilities in
recommender systems. In *Proceedings of the 17th ACM Conference on Recom-
mender Systems*. 1126–1132.
- [14] Markus Freitag and Yaser Al-Onaizan. 2017. Beam Search Strategies for Neural
Machine Translation. In *Proceedings of the First Workshop on Neural Machine
Translation*. 56–60.
- [15] Chongming Gao, Ruijun Chen, Shuai Yuan, Kexin Huang, Yuanqing Yu, and
Xiangnan He. 2025. Sprec: Self-play to debias llm-based recommendation. In
Proceedings of the ACM on Web Conference 2025. 5075–5084.
- [16] Yunfan Gao, Tao Sheng, Youlin Xiang, Yun Xiong, Haofen Wang, and Jiawei
Zhang. 2023. Chat-rec: Towards interactive and explainable llms-augmented
recommender system. *arXiv preprint arXiv:2303.14524* (2023).
- [17] Shijie Geng, Shuchang Liu, Zuohui Fu, Yingqiang Ge, and Yongfeng Zhang. 2022.
Recommendation as language processing (rlp): A unified pretrain, personalized
prompt & predict paradigm (p5). In *Proceedings of the 16th ACM conference on
recommender systems*. 299–315.
- [18] Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozière, David Lopez-Paz, and
Gabriel Synnaeve. 2024. Better & faster large language models via multi-token
prediction. *arXiv preprint arXiv:2404.19737* (2024).
- [19] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek
Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex
Vaughan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*
(2024).
- [20] Ruining He and Julian McAuley. 2016. Ups and downs: Modeling the visual
evolution of fashion trends with one-class collaborative filtering. In *proceedings
of the 25th international conference on world wide web*. 507–517.
- [21] Balázs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk.
2015. Session-based recommendations with recurrent neural networks. *arXiv
preprint arXiv:1511.06939* (2015).
- [22] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long short-term memory. *Neural
computation* 9, 8 (1997), 1735–1780.
- [23] Yupeng Hou, Junjie Zhang, Zihan Lin, Hongyu Lu, Ruobing Xie, Julian McAuley,
and Wayne Xin Zhao. 2024. Large language models are zero-shot rankers for
recommender systems. In *European Conference on Information Retrieval*. Springer,
364–381.
- [24] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean
Wang, Lu Wang, Weizhu Chen, et al. 2022. Lora: Low-rank adaptation of large
language models. *ICLR* 1, 2 (2022), 3.

表 5: 在 BIGRec [3]骨干网络上, 对直接优化目标 (3.1) 的直接方案与 SFT、BEAR 进行的性能比较. (3.1) 需要束搜索, 计算成本较高, 因此实验在两个小规模数据集 (Toy-S 和 Book-S) 上进行, 各采样了 2000 名用户。

方法	Toy-S		Book-S	
	NDCG@10	HR@10	NDCG@10	HR@10
+ SFT	0.0076	0.0200	0.0046	0.0120
+ BEAR	0.0094	0.0260	0.0072	0.0180
+ 对象 (3.1)	0.0103	0.0280	0.0077	0.0200

BEAR 是一种新颖的束搜索感知正则化方法, 它在训练过程中显式地考虑了束搜索行为。BEAR 优化了在束搜索期间保留正样本的必要条件, 从而降低了错误剪枝的风险并提升了推荐性能, 同时与标准 SFT 相比仅产生极小的计算开销。对于未来的研-
方向, 探索将 BEAR 与推理阶段技术 (如多 token 预测 [18] 和多样化束搜索 [55],) 相结合将是一个富有前景的研究途径。

A 附录: 进一步讨论

BEAR 与直接解决方案的对比。正如第3.1节所讨论的, 缓解错误剪枝问题的一个朴素方案是直接优化目标函数 (3.1)。虽然这种直接的方法看似“理论最优”, 但它要求在训练过程中执行束搜索, 这在基于大语言模型的推荐系统中会带来不切实际的计算开销。相反, 我们提出的 BEAR 方法选择优化保留正样本的必要条件, 提供了一种计算效率更高的替代方案。一个关键的研究问题自然随之产生: 与“最优”但昂贵的解决方案相比, 优化这一较弱的必要条件是否会牺牲显著的有效性?

为回答这一问题, 我们在表5中的两个数据集上开展了实验, 将 BEAR 与直接优化目标 (3.1) 进行了对比。观察发现, BEAR 与计算密集的目标 (3.1) 均显著优于标准 SFT, 验证了二者缓解训练 - 推理不一致性的能力。更重要的是, BEAR 与目标 (3.1) 之间的性能差距微乎其微。这表明, BEAR 在捕捉束搜索剪枝的核心动态方面足够有效, 在与直接方案实现性能持平的同时, 避免了其高昂的计算成本。值得注意的是, 这些结果也与我们在图4中的发现完全一致: 由于违反必要条件构成了绝大多数错误剪枝案例, 通过 BEAR 有效针对这一主要原因即可获得大部分潜在增益, 这也进一步解释了其优异的性能表现。

参考文献

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Floren-
cia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal
Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*
(2023).
- [2] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan,
Wenbin Ge, Yu Han, Fei Huang, et al. 2023. Qwen technical report. *arXiv preprint
arXiv:2309.16609* (2023).

- [3] Kebin Bao, Jizhi Zhang, Wenjie Wang, Yang Zhang, Zhengyi Yang, Yanchen
Luo, Chong Chen, Fuli Feng, and Qi Tian. 2025. A bi-step grounding paradigm
for large language models in recommendation systems. *ACM Transactions on
Recommender Systems* 3, 4 (2025), 1–27.
- [4] Kebin Bao, Jizhi Zhang, Yang Zhang, Xinyue Huo, Chong Chen, and Fuli Feng.
2024. Decoding matters: Addressing amplification bias and homogeneity issue in
recommendations for large language models. In *Proceedings of the 2024 Conference
on Empirical Methods in Natural Language Processing*. 10540–10552.
- [5] Kebin Bao, Jizhi Zhang, Yang Zhang, Wenjie Wang, Fuli Feng, and Xiangnan
He. 2023. Tallrec: An effective and efficient tuning framework to align large
language model with recommendation. In *Proceedings of the 17th ACM conference
on recommender systems*. 1007–1014.
- [6] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan,
Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda
Askell, et al. 2020. Language models are few-shot learners. *Advances in neural
information processing systems* 33 (2020), 1877–1901.
- [7] Guohao Cai, Jieming Zhu, Quanyu Dai, Zhenhua Dong, Xiuqiang He, Ruiming
Tang, and Rui Zhang. 2022. Reloop: A self-correction continual learning loop
for recommender systems. In *Proceedings of the 45th International ACM SIGIR
Conference on Research and Development in Information Retrieval*. 2692–2697.
- [8] Jianxin Chang, Chen Gao, Yu Zheng, Yiqun Hui, Yanan Niu, Yang Song, Depeng
Jin, and Yong Li. 2021. Sequential recommendation with graph neural networks.
In *Proceedings of the 44th international ACM SIGIR conference on research and
development in information retrieval*. 378–387.
- [9] Xu Chen, Hongteng Xu, Yongfeng Zhang, Jiayi Tang, Yixin Cao, Zheng Qin, and
Hongyuan Zha. 2018. Sequential recommendation with user memory networks.
In *Proceedings of the eleventh ACM international conference on web search and
data mining*. 108–116.
- [10] Yuxin Chen, Junfei Tan, An Zhang, Zhengyi Yang, Leheng Sheng, Enzhi Zhang,
Xiang Wang, and Tat-Seng Chua. 2024. On softmax direct preference optimization
for recommendation. *Advances in Neural Information Processing Systems* 37 (2024),
27463–27489.
- [11] Yu Cui, Feng Liu, Pengbo Wang, Bohao Wang, Heng Tang, Yi Wan, Jun Wang, and
Jiawei Chen. 2024. Distillation matters: empowering sequential recommenders
to match the performance of large language models. In *Proceedings of the 18th
ACM Conference on Recommender Systems*. 507–517.
- [12] Zeyu Cui, Jianxin Ma, Chang Zhou, Jingren Zhou, and Hongxia Yang. 2022.
M6-rec: Generative pretrained language models are open-ended recommender
systems. *arXiv preprint arXiv:2205.08084* (2022).
- [13] Sunhao Dai, Ninglu Shao, Haiyuan Zhao, Weijie Yu, Zihua Si, Chen Xu, Zhongx-
iang Sun, Xiao Zhang, and Jun Xu. 2023. Uncovering chatgpt’s capabilities in
recommender systems. In *Proceedings of the 17th ACM Conference on Recom-
mender Systems*. 1126–1132.
- [14] Markus Freitag and Yaser Al-Onaizan. 2017. Beam Search Strategies for Neural
Machine Translation. In *Proceedings of the First Workshop on Neural Machine
Translation*. 56–60.
- [15] Chongming Gao, Ruijun Chen, Shuai Yuan, Kexin Huang, Yuanqing Yu, and
Xiangnan He. 2025. Sprec: Self-play to debias llm-based recommendation. In
Proceedings of the ACM on Web Conference 2025. 5075–5084.
- [16] Yunfan Gao, Tao Sheng, Youlin Xiang, Yun Xiong, Haofen Wang, and Jiawei
Zhang. 2023. Chat-rec: Towards interactive and explainable llms-augmented
recommender system. *arXiv preprint arXiv:2303.14524* (2023).
- [17] Shijie Geng, Shuchang Liu, Zuohui Fu, Yingqiang Ge, and Yongfeng Zhang. 2022.
Recommendation as language processing (rlp): A unified pretrain, personalized
prompt & predict paradigm (p5). In *Proceedings of the 16th ACM conference on
recommender systems*. 299–315.
- [18] Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozière, David Lopez-Paz, and
Gabriel Synnaeve. 2024. Better & faster large language models via multi-token
prediction. *arXiv preprint arXiv:2404.19737* (2024).
- [19] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek
Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex
Vaughan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*
(2024).
- [20] Ruining He and Julian McAuley. 2016. Ups and downs: Modeling the visual
evolution of fashion trends with one-class collaborative filtering. In *proceedings
of the 25th international conference on world wide web*. 507–517.
- [21] Balázs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk.
2015. Session-based recommendations with recurrent neural networks. *arXiv
preprint arXiv:1511.06939* (2015).
- [22] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long short-term memory. *Neural
computation* 9, 8 (1997), 1735–1780.
- [23] Yupeng Hou, Junjie Zhang, Zihan Lin, Hongyu Lu, Ruobing Xie, Julian McAuley,
and Wayne Xin Zhao. 2024. Large language models are zero-shot rankers for
recommender systems. In *European Conference on Information Retrieval*. Springer,
364–381.
- [24] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean
Wang, Lu Wang, Weizhu Chen, et al. 2022. Lora: Low-rank adaptation of large
language models. *ICLR* 1, 2 (2022), 3.

- [25] Wang-Cheng Kang and Julian McAuley. 2018. Self-attentive sequential recommendation. In *2018 IEEE international conference on data mining (ICDM)*. IEEE, 197–206.
- [26] Sein Kim, Hongseok Kang, Seungyeon Choi, Donghyun Kim, Minchul Yang, and Chanyoung Park. 2024. Large language models meet collaborative filtering: An efficient all-round llm-based recommender system. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 1395–1406.
- [27] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. 2012. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems* 25 (2012).
- [28] Jiacheng Li, Yujie Wang, and Julian McAuley. 2020. Time interval aware self-attention for sequential recommendation. In *Proceedings of the 13th international conference on web search and data mining*. 322–330.
- [29] Xiaoxi Li, Jiajie Jin, Yujia Zhou, Yuyao Zhang, Peitian Zhang, Yutao Zhu, and Zhicheng Dou. 2025. From matching to generation: A survey on generative information retrieval. *ACM Transactions on Information Systems* 43, 3 (2025), 1–62.
- [30] Xinyi Li, Yongfeng Zhang, and Edward C Malthouse. 2023. Exploring fine-tuning chatgpt for news recommendation. *arXiv preprint arXiv:2311.05850* (2023).
- [31] Jiayi Liao, Xiangnan He, Ruobing Xie, Jiancan Wu, Yancheng Yuan, Xingwu Sun, Zhanhui Kang, and Xiang Wang. 2024. Rosepo: Aligning llm-based recommenders with human values. *arXiv preprint arXiv:2410.12519* (2024).
- [32] Jiayi Liao, Sihang Li, Zhengyi Yang, Jiancan Wu, Yancheng Yuan, Xiang Wang, and Xiangnan He. 2024. Llara: Large language-recommendation assistant. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1785–1795.
- [33] Zijie Lin, Yang Zhang, Xiaoyan Zhao, Fengbin Zhu, Fuli Feng, and Tat-Seng Chua. 2025. IGD: Token Decisiveness Modeling via Information Gain in LLMs for Personalized Recommendation. *arXiv preprint arXiv:2506.13229* (2025).
- [34] Enze Liu, Bowen Zheng, Xiao lei Wang, Wayne Xin Zhao, Jinpeng Wang, Sheng Chen, and Ji-Rong Wen. 2025. LARES: Latent Reasoning for Sequential Recommendation. *arXiv preprint arXiv:2505.16865* (2025).
- [35] Junling Liu, Chao Liu, Peilin Zhou, Renjie Lv, Kang Zhou, and Yan Zhang. 2023. Is chatgpt a good recommender? a preliminary study. *arXiv preprint arXiv:2304.10149* (2023).
- [36] Qijiong Liu, Nuo Chen, Tetsuya Sakai, and Xiao-Ming Wu. 2024. Once: Boosting content-based recommendation with both open-and closed-source large language models. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*. 452–461.
- [37] Ze Liu, Jin Zhang, Chao Feng, Defu Lian, Jie Wang, and Enhong Chen. 2024. Learning Deep Tree-based Retriever for Efficient Recommendation: Theory and Method. *arXiv preprint arXiv:2408.11345* (2024).
- [38] Ilya Loshchilov and Frank Hutter. 2017. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101* (2017).
- [39] Sijin Lu, Zhibo Man, Fangyuan Luo, and Jun Wu. 2025. Dual Debiasing in LLM-based Recommendation. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2685–2689.
- [40] Julian McAuley, Christopher Targett, Qinfeng Shi, and Anton Van Den Hengel. 2015. Image-based recommendations on styles and substitutes. In *Proceedings of the 38th international ACM SIGIR conference on research and development in information retrieval*. 43–52.
- [41] Fei Mi, Xiaoyu Lin, and Boi Faltings. 2020. Ader: Adaptively distilled exemplar replay towards continual learning for session-based recommendation. In *Proceedings of the 14th ACM Conference on Recommender Systems*. 408–413.
- [42] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *Advances in neural information processing systems* 35 (2022), 27730–27744.
- [43] Weizhen Qi, Yeyun Gong, Yu Yan, Jian Jiao, Bo Shao, Ruofei Zhang, Houqiang Li, Nan Duan, and Ming Zhou. 2020. Prophetnet-ads: A looking ahead strategy for generative retrieval models in sponsored search engine. In *CCF International Conference on Natural Language Processing and Chinese Computing*. Springer, 305–317.
- [44] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog* 1, 8 (2019), 9.
- [45] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. 2023. Direct preference optimization: Your language model is secretly a reward model. *Advances in neural information processing systems* 36 (2023), 53728–53741.
- [46] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. SQuAD: 100,000+ Questions for Machine Comprehension of Text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*. 2383–2392.
- [47] Wentao Shi, Xiangnan He, Yang Zhang, Chongming Gao, Xinyue Li, Jizhi Zhang, Qifan Wang, and Fuli Feng. 2024. Large language models are learnable planners for long-term recommendation. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1893–1903.
- [48] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer. In *Proceedings of the 28th ACM international conference on information and knowledge management*. 1441–1450.
- [49] Zhongxiang Sun, Zihua Si, Xiaoxue Zang, Kai Zheng, Yang Song, Xiao Zhang, and Jun Xu. 2024. Large language models enhanced collaborative filtering. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*. 2178–2188.
- [50] Jiakai Tang, Sunhao Dai, Teng Shi, Jun Xu, Xu Chen, Wen Chen, Jian Wu, and Yuning Jiang. 2025. Think before recommend: Unleashing the latent reasoning power for sequential recommendation. *arXiv preprint arXiv:2503.22675* (2025).
- [51] Jiayi Tang and Ke Wang. 2018. Personalized top-n sequential recommendation via convolutional sequence embedding. In *Proceedings of the eleventh ACM international conference on web search and data mining*. 565–573.
- [52] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
- [53] Vladimir Vapnik. 2013. *The nature of statistical learning theory*. Springer science & business media.
- [54] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [55] Ashwin Vijayakumar, Michael Cogswell, Ramprasaath Selvaraju, Qing Sun, Stefan Lee, David Crandall, and Dhruv Batra. 2018. Diverse beam search for improved description of complex scenes. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 32.
- [56] Bohao Wang, Feng Liu, Jiawei Chen, Xingyu Lou, Changwang Zhang, Jun Wang, Yuegang Sun, Yan Feng, Chun Chen, and Can Wang. 2025. Msl: Not all tokens are what you need for tuning llm as a recommender. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1912–1922.
- [57] Bohao Wang, Feng Liu, Changwang Zhang, Jiawei Chen, Yudi Wu, Sheng Zhou, Xingyu Lou, Jun Wang, Yan Feng, Chun Chen, and Can Wang. 2025. LLM4DSR: Leveraging Large Language Model for Denoising Sequential Recommendation. *ACM Transactions on Information Systems* (Aug. 2025). Just Accepted.
- [58] Lei Wang and Ee-Peng Lim. 2023. Zero-shot next-item recommendation using large pretrained language models. *arXiv preprint arXiv:2304.03153* (2023).
- [59] Zhefan Wang, Weizhi Ma, and Min Zhang. 2024. To recommend or not: Recommendation identification in conversations with pre-trained language models. In *International Conference on Database Systems for Advanced Applications*. Springer, 19–35.
- [60] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
- [61] Wei Wei, Chao Huang, Lianghao Xia, and Chuxu Zhang. 2023. Multi-modal self-supervised learning for recommendation. In *Proceedings of the ACM web conference 2023*. 790–800.
- [62] Jiancan Wu, Xiang Wang, Xingyu Gao, Jiawei Chen, Hongcheng Fu, and Tianyu Qiu. 2024. On the effectiveness of sampled softmax loss for item recommendation. *ACM Transactions on Information Systems* 42, 4 (2024), 1–26.
- [63] Shiguang Wu, Zhaochun Ren, Xin Xin, Jiyuan Yang, Mengqi Zhang, Zhumin Chen, Maarten de Rijke, and Pengjie Ren. 2025. Constrained Auto-Regressive Decoding Constrains Generative Retrieval. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2429–2440.
- [64] Xue Xia, Pong Eksombatchai, Nikil Pancha, Dhruvil Deven Badani, Po-Wei Wang, Neng Gu, Saurabh Vishwas Joshi, Nazanin Farahpour, Zhiyuan Zhang, and Andrew Zhai. 2023. Transact: Transformer-based realtime user action model for recommendation at pinterest. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 5249–5259.
- [65] An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, et al. 2024. Qwen2. 5-math technical report: Toward mathematical expert model via self-improvement. *arXiv preprint arXiv:2409.12122* (2024).
- [66] Weiqin Yang, Jiawei Chen, Xin Xin, Sheng Zhou, Binbin Hu, Yan Feng, Chun Chen, and Can Wang. 2024. PSL: Rethinking and Improving Softmax Loss from Pairwise Perspective for Recommendation. *Advances in Neural Information Processing Systems* 37 (2024), 120974–121006.
- [67] Weiqin Yang, Jiawei Chen, Shengjia Zhang, Peng Wu, Yuegang Sun, Yan Feng, Chun Chen, and Can Wang. 2025. Breaking the Top-K Barrier: Advancing Top-K Ranking Metrics Optimization in Recommender Systems. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*. 3542–3552.
- [68] Zhengyi Yang, Xiangnan He, Jizhi Zhang, Jiancan Wu, Xin Xin, Jiawei Chen, and Xiang Wang. 2023. A generic learning framework for sequential recommendation with distribution shifts. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1893–1903.
- [25] Wang-Cheng Kang and Julian McAuley. 2018. Self-attentive sequential recommendation. In *2018 IEEE international conference on data mining (ICDM)*. IEEE, 197–206.
- [26] Sein Kim, Hongseok Kang, Seungyeon Choi, Donghyun Kim, Minchul Yang, and Chanyoung Park. 2024. Large language models meet collaborative filtering: An efficient all-round llm-based recommender system. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 1395–1406.
- [27] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. 2012. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems* 25 (2012).
- [28] Jiacheng Li, Yujie Wang, and Julian McAuley. 2020. Time interval aware self-attention for sequential recommendation. In *Proceedings of the 13th international conference on web search and data mining*. 322–330.
- [29] Xiaoxi Li, Jiajie Jin, Yujia Zhou, Yuyao Zhang, Peitian Zhang, Yutao Zhu, and Zhicheng Dou. 2025. From matching to generation: A survey on generative information retrieval. *ACM Transactions on Information Systems* 43, 3 (2025), 1–62.
- [30] Xinyi Li, Yongfeng Zhang, and Edward C Malthouse. 2023. Exploring fine-tuning chatgpt for news recommendation. *arXiv preprint arXiv:2311.05850* (2023).
- [31] Jiayi Liao, Xiangnan He, Ruobing Xie, Jiancan Wu, Yancheng Yuan, Xingwu Sun, Zhanhui Kang, and Xiang Wang. 2024. Rosepo: Aligning llm-based recommenders with human values. *arXiv preprint arXiv:2410.12519* (2024).
- [32] Jiayi Liao, Sihang Li, Zhengyi Yang, Jiancan Wu, Yancheng Yuan, Xiang Wang, and Xiangnan He. 2024. Llara: Large language-recommendation assistant. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1785–1795.
- [33] Zijie Lin, Yang Zhang, Xiaoyan Zhao, Fengbin Zhu, Fuli Feng, and Tat-Seng Chua. 2025. IGD: Token Decisiveness Modeling via Information Gain in LLMs for Personalized Recommendation. *arXiv preprint arXiv:2506.13229* (2025).
- [34] Enze Liu, Bowen Zheng, Xiao lei Wang, Wayne Xin Zhao, Jinpeng Wang, Sheng Chen, and Ji-Rong Wen. 2025. LARES: Latent Reasoning for Sequential Recommendation. *arXiv preprint arXiv:2505.16865* (2025).
- [35] Junling Liu, Chao Liu, Peilin Zhou, Renjie Lv, Kang Zhou, and Yan Zhang. 2023. Is chatgpt a good recommender? a preliminary study. *arXiv preprint arXiv:2304.10149* (2023).
- [36] Qijiong Liu, Nuo Chen, Tetsuya Sakai, and Xiao-Ming Wu. 2024. Once: Boosting content-based recommendation with both open-and closed-source large language models. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*. 452–461.
- [37] Ze Liu, Jin Zhang, Chao Feng, Defu Lian, Jie Wang, and Enhong Chen. 2024. Learning Deep Tree-based Retriever for Efficient Recommendation: Theory and Method. *arXiv preprint arXiv:2408.11345* (2024).
- [38] Ilya Loshchilov and Frank Hutter. 2017. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101* (2017).
- [39] Sijin Lu, Zhibo Man, Fangyuan Luo, and Jun Wu. 2025. Dual Debiasing in LLM-based Recommendation. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2685–2689.
- [40] Julian McAuley, Christopher Targett, Qinfeng Shi, and Anton Van Den Hengel. 2015. Image-based recommendations on styles and substitutes. In *Proceedings of the 38th international ACM SIGIR conference on research and development in information retrieval*. 43–52.
- [41] Fei Mi, Xiaoyu Lin, and Boi Faltings. 2020. Ader: Adaptively distilled exemplar replay towards continual learning for session-based recommendation. In *Proceedings of the 14th ACM Conference on Recommender Systems*. 408–413.
- [42] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *Advances in neural information processing systems* 35 (2022), 27730–27744.
- [43] Weizhen Qi, Yeyun Gong, Yu Yan, Jian Jiao, Bo Shao, Ruofei Zhang, Houqiang Li, Nan Duan, and Ming Zhou. 2020. Prophetnet-ads: A looking ahead strategy for generative retrieval models in sponsored search engine. In *CCF International Conference on Natural Language Processing and Chinese Computing*. Springer, 305–317.
- [44] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog* 1, 8 (2019), 9.
- [45] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. 2023. Direct preference optimization: Your language model is secretly a reward model. *Advances in neural information processing systems* 36 (2023), 53728–53741.
- [46] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. SQuAD: 100,000+ Questions for Machine Comprehension of Text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*. 2383–2392.
- [47] Wentao Shi, Xiangnan He, Yang Zhang, Chongming Gao, Xinyue Li, Jizhi Zhang, Qifan Wang, and Fuli Feng. 2024. Large language models are learnable planners for long-term recommendation. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1893–1903.
- [48] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer. In *Proceedings of the 28th ACM international conference on information and knowledge management*. 1441–1450.
- [49] Zhongxiang Sun, Zihua Si, Xiaoxue Zang, Kai Zheng, Yang Song, Xiao Zhang, and Jun Xu. 2024. Large language models enhanced collaborative filtering. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*. 2178–2188.
- [50] Jiakai Tang, Sunhao Dai, Teng Shi, Jun Xu, Xu Chen, Wen Chen, Jian Wu, and Yuning Jiang. 2025. Think before recommend: Unleashing the latent reasoning power for sequential recommendation. *arXiv preprint arXiv:2503.22675* (2025).
- [51] Jiayi Tang and Ke Wang. 2018. Personalized top-n sequential recommendation via convolutional sequence embedding. In *Proceedings of the eleventh ACM international conference on web search and data mining*. 565–573.
- [52] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
- [53] Vladimir Vapnik. 2013. *The nature of statistical learning theory*. Springer science & business media.
- [54] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [55] Ashwin Vijayakumar, Michael Cogswell, Ramprasaath Selvaraju, Qing Sun, Stefan Lee, David Crandall, and Dhruv Batra. 2018. Diverse beam search for improved description of complex scenes. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 32.
- [56] Bohao Wang, Feng Liu, Jiawei Chen, Xingyu Lou, Changwang Zhang, Jun Wang, Yuegang Sun, Yan Feng, Chun Chen, and Can Wang. 2025. Msl: Not all tokens are what you need for tuning llm as a recommender. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1912–1922.
- [57] Bohao Wang, Feng Liu, Changwang Zhang, Jiawei Chen, Yudi Wu, Sheng Zhou, Xingyu Lou, Jun Wang, Yan Feng, Chun Chen, and Can Wang. 2025. LLM4DSR: Leveraging Large Language Model for Denoising Sequential Recommendation. *ACM Transactions on Information Systems* (Aug. 2025). Just Accepted.
- [58] Lei Wang and Ee-Peng Lim. 2023. Zero-shot next-item recommendation using large pretrained language models. *arXiv preprint arXiv:2304.03153* (2023).
- [59] Zhefan Wang, Weizhi Ma, and Min Zhang. 2024. To recommend or not: Recommendation identification in conversations with pre-trained language models. In *International Conference on Database Systems for Advanced Applications*. Springer, 19–35.
- [60] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
- [61] Wei Wei, Chao Huang, Lianghao Xia, and Chuxu Zhang. 2023. Multi-modal self-supervised learning for recommendation. In *Proceedings of the ACM web conference 2023*. 790–800.
- [62] Jiancan Wu, Xiang Wang, Xingyu Gao, Jiawei Chen, Hongcheng Fu, and Tianyu Qiu. 2024. On the effectiveness of sampled softmax loss for item recommendation. *ACM Transactions on Information Systems* 42, 4 (2024), 1–26.
- [63] Shiguang Wu, Zhaochun Ren, Xin Xin, Jiyuan Yang, Mengqi Zhang, Zhumin Chen, Maarten de Rijke, and Pengjie Ren. 2025. Constrained Auto-Regressive Decoding Constrains Generative Retrieval. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2429–2440.
- [64] Xue Xia, Pong Eksombatchai, Nikil Pancha, Dhruvil Deven Badani, Po-Wei Wang, Neng Gu, Saurabh Vishwas Joshi, Nazanin Farahpour, Zhiyuan Zhang, and Andrew Zhai. 2023. Transact: Transformer-based realtime user action model for recommendation at pinterest. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 5249–5259.
- [65] An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, et al. 2024. Qwen2. 5-math technical report: Toward mathematical expert model via self-improvement. *arXiv preprint arXiv:2409.12122* (2024).
- [66] Weiqin Yang, Jiawei Chen, Xin Xin, Sheng Zhou, Binbin Hu, Yan Feng, Chun Chen, and Can Wang. 2024. PSL: Rethinking and Improving Softmax Loss from Pairwise Perspective for Recommendation. *Advances in Neural Information Processing Systems* 37 (2024), 120974–121006.
- [67] Weiqin Yang, Jiawei Chen, Shengjia Zhang, Peng Wu, Yuegang Sun, Yan Feng, Chun Chen, and Can Wang. 2025. Breaking the Top-K Barrier: Advancing Top-K Ranking Metrics Optimization in Recommender Systems. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*. 3542–3552.
- [68] Zhengyi Yang, Xiangnan He, Jizhi Zhang, Jiancan Wu, Xin Xin, Jiawei Chen, and Xiang Wang. 2023. A generic learning framework for sequential recommendation with distribution shifts. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1893–1903.

Conference on Research and Development in Information Retrieval. 331–340.

[69] Hansi Zeng, Chen Luo, Bowen Jin, Sheikh Muhammad Sarwar, Tianxin Wei, and Hamed Zamani. 2024. Scalable and effective generative information retrieval. In *Proceedings of the ACM Web Conference 2024*. 1441–1452.

[70] Hansi Zeng, Chen Luo, and Hamed Zamani. 2024. Planning ahead in generative retrieval: Guiding autoregressive generation through simultaneous decoding. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 469–480.

[71] Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao, Leon Gao, Zhaojie Gong, Fangda Gu, Michael He, et al. 2024. Actions speak louder than words: Trillion-parameter sequential transducers for generative recommendations. *arXiv preprint arXiv:2402.17152* (2024).

[72] Junjie Zhang, Ruobing Xie, Yupeng Hou, Xin Zhao, Leyu Lin, and Ji-Rong Wen. 2025. Recommendation as instruction following: A large language model empowered recommendation approach. *ACM Transactions on Information Systems*

43, 5 (2025), 1–37.

[73] Yang Zhang, Juntao You, Yimeng Bai, Jizhi Zhang, Keqin Bao, Wenjie Wang, and Tat-Seng Chua. 2024. Causality-enhanced behavior sequence modeling in LLMs for personalized recommendation. *arXiv preprint arXiv:2410.22809* (2024).

[74] Xin Zhou, Hongyu Zhou, Yong Liu, Zhiwei Zeng, Chunyan Miao, Pengwei Wang, Yuan You, and Feijun Jiang. 2023. Bootstrap latent representations for multi-modal recommendation. In *Proceedings of the ACM web conference 2023*. 845–854.

[75] Qihao Zhu, Daya Guo, Zhihong Shao, Dejian Yang, Peiyi Wang, Runxin Xu, Y Wu, Yukun Li, Huazuo Gao, Shirong Ma, et al. 2024. Deepseek-coder-v2: Breaking the barrier of closed-source models in code intelligence. *arXiv preprint arXiv:2406.11931* (2024).

[76] Jingwei Zhuo, Ziru Xu, Wei Dai, Han Zhu, Han Li, Jian Xu, and Kun Gai. 2020. Learning optimal tree models under beam search. In *International Conference on Machine Learning*. PMLR, 11650–11659.

Conference on Research and Development in Information Retrieval. 331–340.

[69] Hansi Zeng, Chen Luo, Bowen Jin, Sheikh Muhammad Sarwar, Tianxin Wei, and Hamed Zamani. 2024. Scalable and effective generative information retrieval. In *Proceedings of the ACM Web Conference 2024*. 1441–1452.

[70] Hansi Zeng, Chen Luo, and Hamed Zamani. 2024. Planning ahead in generative retrieval: Guiding autoregressive generation through simultaneous decoding. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 469–480.

[71] Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao, Leon Gao, Zhaojie Gong, Fangda Gu, Michael He, et al. 2024. Actions speak louder than words: Trillion-parameter sequential transducers for generative recommendations. *arXiv preprint arXiv:2402.17152* (2024).

[72] Junjie Zhang, Ruobing Xie, Yupeng Hou, Xin Zhao, Leyu Lin, and Ji-Rong Wen. 2025. Recommendation as instruction following: A large language model empowered recommendation approach. *ACM Transactions on Information Systems*

43, 5 (2025), 1–37.

[73] Yang Zhang, Juntao You, Yimeng Bai, Jizhi Zhang, Keqin Bao, Wenjie Wang, and Tat-Seng Chua. 2024. Causality-enhanced behavior sequence modeling in LLMs for personalized recommendation. *arXiv preprint arXiv:2410.22809* (2024).

[74] Xin Zhou, Hongyu Zhou, Yong Liu, Zhiwei Zeng, Chunyan Miao, Pengwei Wang, Yuan You, and Feijun Jiang. 2023. Bootstrap latent representations for multi-modal recommendation. In *Proceedings of the ACM web conference 2023*. 845–854.

[75] Qihao Zhu, Daya Guo, Zhihong Shao, Dejian Yang, Peiyi Wang, Runxin Xu, Y Wu, Yukun Li, Huazuo Gao, Shirong Ma, et al. 2024. Deepseek-coder-v2: Breaking the barrier of closed-source models in code intelligence. *arXiv preprint arXiv:2406.11931* (2024).

[76] Jingwei Zhuo, Ziru Xu, Wei Dai, Han Zhu, Han Li, Jian Xu, and Kun Gai. 2020. Learning optimal tree models under beam search. In *International Conference on Machine Learning*. PMLR, 11650–11659.